

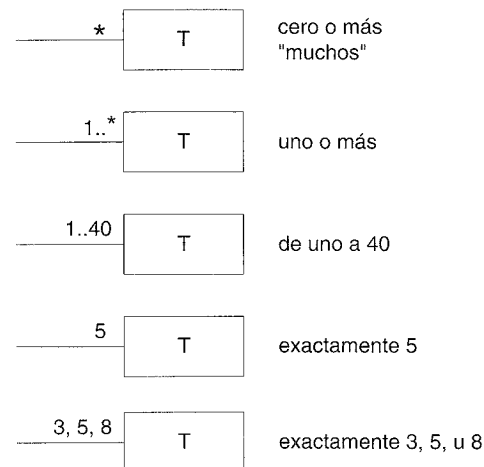
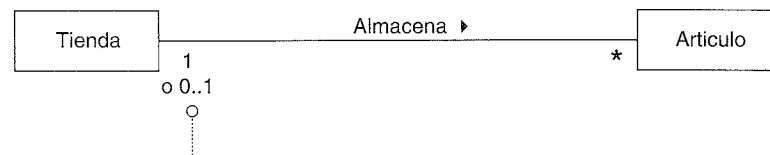


Figura 11.4. Valores de la multiplicidad.



¿La multiplicidad debería ser "1" o "0..1"?

La respuesta depende de nuestro interés al utilizar el modelo. Típica y prácticamente, la multiplicidad denota una restricción del dominio que nos preocupamos de ser capaces de comprobar en el software, si esta relación se implementase o reflejase en los objetos software o en la base de datos. Por ejemplo, un concreto artículo podría llegar a venderse o desecharse, y por tanto, no se almacenaría en la tienda por más tiempo. Desde este punto de vista, "0..1" es lógico, pero...

¿Nos preocupa este punto de vista? Si se implementa esta relación en el software, probablemente querríamos asegurar que una instancia software de *Articulo* siempre se relacionara con una instancia concreta de *Tienda*, de otra manera, indica un error o corrupción en los elementos software o datos.

Este modelo de dominio parcial no representa objetos software, sino que las multiplicidades registran restricciones cuyo valor práctico está, normalmente, relacionado con nuestro interés en la construcción del software o bases de datos (que reflejan nuestro dominio del mundo real) con comprobaciones de validez. Desde este punto de vista, el valor deseable podría ser "1".

Figura 11.5. La multiplicidad es dependiente del contexto.

Rumbaugh proporciona otro ejemplo de *Persona* y *Compañia* en la asociación *Trabaja-para* [Rumbaugh91]. El que indiquemos si una instancia de *Persona* trabaja para una o varias instancias de *Compañia*, es dependiente del contexto del modelo; el departamento de impuestos está interesado en *muchas*; un sindicato probablemente sólo en *una*. Normalmente, la elección, prácticamente, depende de para quién estamos construyendo el software, y de esta manera, las multiplicidades válidas de una implementación.

11.6. ¿Cómo de detalladas deben ser las asociaciones?

Las asociaciones son importantes, pero un error típico al crear los modelos del dominio, es dedicar demasiado tiempo durante el estudio intentando descubrirlas.

Es fundamental entender lo siguiente:

Es más importante encontrar las *clases conceptuales* que las asociaciones. La mayoría del tiempo dedicado a la creación del modelo del dominio debería emplearse en la identificación de las clases conceptuales, no de las asociaciones.

11.7. Asignación de nombres a las asociaciones

Nombre una asociación en base al formato *NombreTipo-FraseVerbal-NombreTipo* donde la frase verbal crea una secuencia que es legible y tiene significado en el contexto del modelo.

Los nombres de las asociaciones deben comenzar con una letra mayúscula, puesto que una asociación representa un clasificador de enlace entre las instancias; en UML, los clasificadores deben comenzar con una letra mayúscula. Dos formatos típicos e igualmente válidos para un nombre de asociación compuesta son:

- *Pagado-mediante*.
- *PagadoMediante*.

En la Figura 11.6, la dirección por defecto para la lectura de los nombres de las asociaciones es de izquierda a derecha o de arriba abajo. No se corresponde con la dirección por defecto en UML, sino que se trata de una convención común.

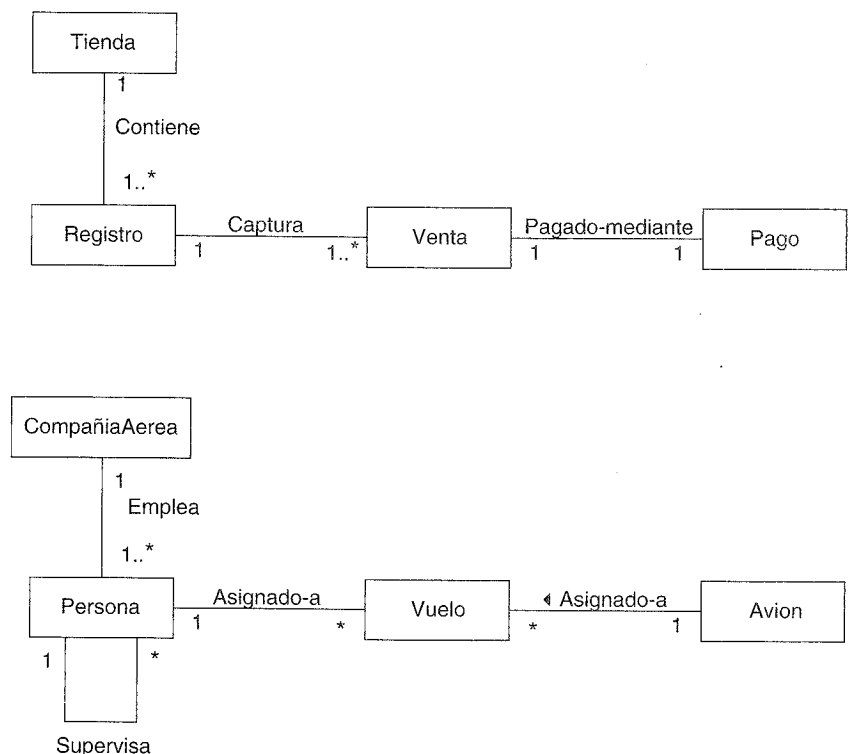


Figura 11.6. Nombres de asociaciones.

11.8. Múltiples asociaciones entre dos tipos

Dos tipos podrían tener múltiples asociaciones entre ellos; no es extraño. No existe ningún ejemplo destacado en nuestro caso de estudio del PDV, pero un ejemplo del dominio de la compañía aérea podría ser las relaciones entre un *Vuelo* (o quizás de manera más precisa, una *EtapaVuelo*) y un *Aeropuerto* (ver Figura 11.7); las asociaciones vuela-a y vuela-desde son relaciones claramente diferentes, que se deberían mostrar de manera separada.

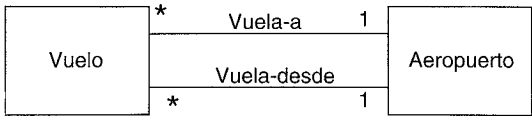


Figura 11.7. Múltiples asociaciones.

11.9. Asociaciones e implementación

Durante el modelado del dominio, una asociación *no* es una declaración sobre el flujo de datos, variables de instancia o conexiones entre objetos en una solución software; es una manifestación de que una relación es significativa en un sentido puramente conceptual —en el mundo real—. Desde un punto de vista práctico, muchas de estas relaciones se implementarán generalmente en software como caminos de navegación y visibilidad (tanto en el Modelo de Diseño como en el Modelo de Datos), pero su presencia en una vista conceptual (o esencial) de un modelo de dominio no requiere su implementación.

Al crear un modelo de dominio, podríamos definir asociaciones que no son necesarias durante la implementación. A la inversa, podríamos descubrir asociaciones que necesitan implementarse pero que se obviaron durante el modelado del dominio. En estos casos, el modelo del dominio puede actualizarse para reflejar estos descubrimientos.

Sugerencia

¿Deberían actualizarse los anteriores modelos estudiados, como el modelo del dominio, con las apreciaciones descubiertas durante el trabajo de implementación? No se moleste a menos que haya algún uso práctico del modelo en el futuro. Si es únicamente (como es el caso algunas veces) un artefacto temporal que se utiliza para inspirar las etapas siguientes, y no se utilizará de manera significativa más adelante, ¿por qué actualizarlo? Evite hacer o actualizar cualquier modelo o documentación a menos que exista una justificación concreta para su futura utilización.

Posteriormente estudiaremos formas de implementar la asociaciones en un lenguaje de programación orientado a objetos (los más habitual es utilizar un atributo que referencia una instancia de la clase asociada), pero de momento, es conveniente pensar en ellas como expresiones puramente conceptuales, *no* declaraciones sobre la solución de base de datos o software. Como siempre, al posponer las consideraciones de diseño se nos libera de informaciones y decisiones extrañas mientras hacemos un estudio de “análisis” puro y maximiza nuestras opciones de diseño más adelante.

11.10. Asociaciones del Modelo del Dominio del PDV NuevaEra

Ahora podemos añadir las asociaciones a nuestro modelo del dominio del PDV. Deberíamos añadir aquellas asociaciones que los requisitos (por ejemplo, los casos de uso) sugieren o implican una necesidad de recordar, o que, de otra manera, recomienda fuertemente nuestra percepción del dominio del problema. Cuando abordamos un nuevo problema, las categorías comunes de asociaciones, presentadas anteriormente, deberían revisarse y tenerse en cuenta, puesto que representan muchas de las asociaciones relevantes que normalmente necesitan recogerse.

Relaciones evidentes de la Tienda

La siguiente muestra de asociaciones es justificable en términos de necesito-conocer. Se basa en los casos de uso que se están considerando actualmente.

<i>Registro Registra Venta</i>	Para conocer la venta actual, generar el total, imprimir recibo
<i>Venta Pagada-mediante Pago</i>	Para conocer si se ha pagado la venta, relacionar la cantidad entregada con el total de la venta, e imprimir un recibo
<i>CatalogoDeProductos Registra EspecificacionDelProducto</i>	Para recuperar una <i>Especificacion DelProducto</i> a partir de un articuloID

Aplicación de la lista de comprobación de categorías de asociaciones

Recorreremos la lista de comprobación, basada en los tipos identificados previamente, considerando los requisitos del caso de uso actual.

<i>Categoría</i>	<i>Sistema</i>
A es una parte física de B	<i>Registro—Caja</i>
A es una parte lógica de B	<i>LineaDeVenta—Venta</i>
A está contenido físicamente en B	<i>Registo—Tienda</i> <i>Articulo—Tienda</i>
A está contenido lógicamente en B	<i>EspecificacionDelProducto—Catalogo-DeProductos</i> <i>CatalogoDeProductos—Tienda</i>
A es una descripción de B	<i>EspecificacionDelProducto-Articulo</i>

continúa

Nótese que la capacidad de justificar una asociación en función de necesito-conocer depende de los requisitos; obviamente, un cambio en éstos —como que se necesite mostrar en el recibo el ID del cajero— cambia la necesidad de recordar una relación.

Basado en el análisis anterior, *podría* justificarse la eliminación de las asociaciones en cuestión.

Asociaciones necesito-conocer vs. comprensión

Un criterio necesito-conocer estricto para el mantenimiento de las asociaciones generará un “modelo de información” mínimo de lo que se necesita para modelar el dominio del problema —limitado por los requisitos actuales que se están considerando—. Sin embargo, este enfoque podría crear un modelo que no transmite (a nosotros o a los demás) una comprensión completa del dominio.

Además de ser un modelo necesito-conocer de información sobre las cosas, el modelo del dominio es una herramienta de comunicación con la que estamos intentando entender y comunicar a otros los conceptos importantes y sus relaciones. Desde este punto de vista, eliminando algunas asociaciones que no se exigen estrictamente en una base necesito-conocer, puede crear un modelo sin interés —no comunica las ideas claves y relaciones—.

Por ejemplo, en la aplicación del PDV: aunque, tomando como base las relaciones necesito-conocer de manera estricta, podría no ser necesario registrar *Venta Iniciada-por Cliente*, su ausencia deja fuera un aspecto importante para entender el dominio —que un cliente genera las ventas—.

En cuanto a las asociaciones, un buen modelo se sitúa en alguna parte entre un modelo necesito-conocer mínimo y uno que ilustra cada relación concebible. ¿El criterio básico para juzgar su valor? —¿Satisface todos los requisitos necesito-conocer y además comunica claramente un conocimiento esencial de los conceptos importantes en el dominio del problema?—.

Céntrese en las asociaciones necesito-conocer, pero contemple las asociaciones de sólo-comprensión para enriquecer el conocimiento básico del dominio.

MODELO DEL DOMINIO: AÑADIR ATRIBUTOS

Cualquier error tardío es indistinguible de una característica.

Rich Kulawiec

Objetivos

- Identificar los atributos del modelo del dominio.
- Distinguir entre atributos correctos e incorrectos.

Introducción

Resulta útil identificar aquellos atributos de las clases conceptuales que se necesitan para satisfacer los requisitos de información de los actuales escenarios en estudio. Este capítulo explora la identificación de los atributos adecuados, y añade los atributos al modelo del dominio de NuevaEra.

12.1. Atributos

Un **atributo** es un valor de datos lógico de un objeto.

Incluya los siguientes atributos en un modelo del dominio: aquellos para los que los requisitos (por ejemplo, los casos de uso) sugieren o implican una necesidad de registrar la información.

Por ejemplo, un recibo (que recoge la información de una venta) normalmente incluye una fecha y una hora, y la dirección quiere conocer las fechas y horas de las ventas por múltiples motivos. En consecuencia, la clase conceptual *Venta* necesita los atributos *fecha* y *hora*.

12.2. Notación de los atributos en UML

Los atributos se muestran en el segundo compartimento del rectángulo de la clase (ver Figura 12.1). Sus tipos podrían mostrarse opcionalmente.

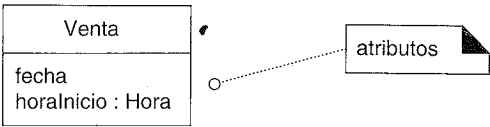


Figura 12.1. Clases y atributos.

12.3. Tipos de atributos válidos

Hay algunas cosas que no deberían representarse como atributos, sino como asociaciones. Esta sección presenta los tipos válidos.

Mantenga atributos simples

Intuitivamente, la mayoría de los atributos simples son los que, a menudo, se conocen como tipos de datos primitivos, como los números. El tipo de un atributo, normalmente, no debería ser un concepto de dominio complejo, como *Venta* o *Aeropuerto*. Por ejemplo, el siguiente atributo *registroActual* en la clase *Cajero* de la Figura 12.2, no es deseable porque su tipo tiene la intención de ser un *Registro*, que no es un tipo de atributo simple (como *Numero* o *String*). La manera más útil para expresar que un *Cajero* utiliza un *Registro* es con una asociación, no con un atributo.

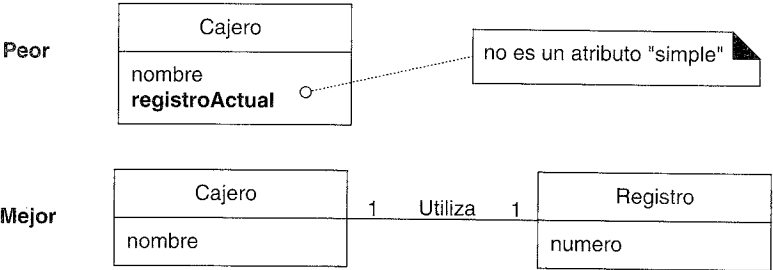


Figura 12.2. Relaciones con asociaciones, no atributos.

Los atributos en un modelo del dominio deberían ser, preferiblemente, **atributos simples** o **tipos de datos**. Los tipos de datos de los atributos muy comunes incluyen: *Boolean*, *Fecha*, *Numero*, *String (Texto)*, *Hora*.

Otros tipos comunes comprenden: *Direccion*, *Color*, *Geometrico (Punto, Rectangulo)*, *Numero de Telefono*, *Numero de la Seguridad Social*, *Codigo de Producto Universal (UPC; Universal Product Code)*, *SKU*, *ZIP* o *códigos postales*, *tipos enumerados*.

Repitiendo un ejemplo previo, un error típico es modelar un concepto del dominio complejo como un atributo. Para ilustrarlo, un aeropuerto de destino no es realmente una cadena de texto; se trata de una cosa compleja que ocupa muchos kilómetros cuadrados de espacio. Por tanto, *Vuelo* debería relacionarse con *Aeropuerto* mediante una asociación, no con un atributo, como se muestra en la Figura 12.3.

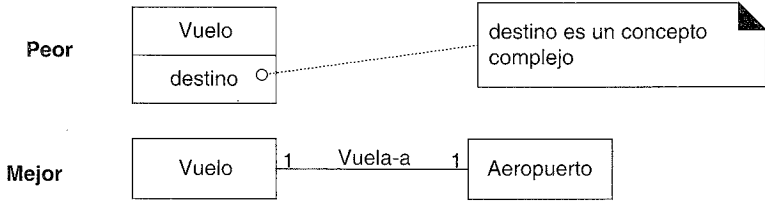


Figura 12.3. Evite la representación de conceptos del dominio complejos como atributos; utilice asociaciones.

Relacione las clases conceptuales con una asociación, no con un atributo.

Perspectiva conceptual vs. implementación: ¿qué sucede con los atributos en el código?

La restricción de que el tipo de los atributos en el modelo del dominio sea sólo un tipo de datos simple *no* implica que los atributos C++ o Java (miembros de datos, campos de una instancia) sólo deban ser de tipos de datos primitivos, o simples. El modelo del dominio se centra en declaraciones conceptuales puras sobre un dominio del problema, no en componentes software.

Posteriormente, durante el trabajo de diseño e implementación, se verá que las asociaciones entre objetos representadas en el modelo del dominio, a menudo, se implementarán como atributos que referencian a otros objetos software complejos. Sin embargo, ésta es sólo una de las posibles soluciones de diseño para implementar una asociación, y, de ahí que la decisión se deba posponer durante el modelado del dominio.

Tipos de datos

Los atributos deben ser, generalmente, **tipos de datos**. Esto es un término UML que implica un conjunto de valores para los cuales no es significativa una identidad única (en el contexto de nuestro modelo o sistema) [RJB99]. Por ejemplo, no es (normalmente) significativo distinguir entre:

- Diferentes instancias del *Numero* 5.
- Diferentes instancias del *String* 'gato'.

- Diferentes instancias del *NumeroDeTelefono* que contiene el mismo número.
- Diferentes instancias de la *Direccion* que contiene la misma dirección.

Por el contrario, es significativo distinguir (por identidad) entre dos instancias de *Persona* cuyos nombres son, en los dos casos, “Luis García”, puesto que las dos instancias pueden representar individuos diferentes con el mismo nombre.

En cuanto al software, existen pocas situaciones donde uno compararía las direcciones de memoria de las instancias de *Numero*, *String*, *NumeroDeTelefono* o *Direccion*; sólo son relevantes las comparaciones basadas en los valores. Por el contrario, es comprensible comparar las direcciones de memoria de las instancias de *Persona*, y distinguirlas, incluso si tienen los mismos valores de los atributos, porque es importante su identidad única.

Por tanto, todos los tipos primitivos (número, string) son tipos de datos UML, pero no todos los tipos de datos son primitivos. Por ejemplo, *NumeroDeTelefono* es un tipo de dato no primitivo.

Estos valores de tipos de datos también se conocen como **objetos valor**.

La noción de tipos de datos puede ser sutil. Como regla empírica, sea fiel a la prueba básica de tipos de atributos “simples”: hágalo un atributo si se considera de manera natural como un número, string, booleano, fecha u hora (etcétera); en otro caso, representelo como una clase conceptual aparte.

En caso de duda, defina algo como una clase conceptual aparte en lugar de como un atributo.

12.4. Clases de tipos de datos no primitivos

El tipo de un atributo podría representarse como una clase no primitiva por derecho propio en un modelo del dominio. Por ejemplo, en el sistema de PDV hay un identificador de artículo. Generalmente se ve simplemente como un número. Así que, ¿se debería representar como una clase no primitiva? Aplique esta guía:

- Represente lo que podría considerarse, inicialmente, como un tipo de dato primitivo (como un número o string) como una clase no primitiva si:
- Está compuesto de secciones separadas.
 - Número de teléfono, nombre de persona.
 - Habitualmente, hay operaciones asociadas con él, como análisis sintáctico o validación
 - Número de la seguridad social.
 - Tiene otros atributos.
 - el precio de promoción podría tener una fecha (efectiva) de comienzo y fin.
 - Es una cantidad con una unidad.

- La cantidad del pago tiene una unidad monetaria.
- Es una abstracción de uno o más tipos con alguna de estas cualidades.
 - El identificador del artículo en el dominio de ventas es una generalización de tipos como el Código de Producto Universal (UPC) o el Número de Artículo Europeo (EAN)

Aplicando estas guías a los atributos del modelo del dominio del PDV llegamos al siguiente análisis:

- El identificador del artículo es una abstracción de varios esquemas de codificación comunes, incluyendo UPC-A, UPC-E y la familia de esquemas EAN. Estos esquemas de codificación numéricos tienen subpartes que identifican al fabricante, producto, país (para EAN), y un dígito de control para validarlos. Por tanto, debería existir una clase no primitiva *ArticuloID*, puesto que satisface muchas de las guías anteriores.
- Los atributos del *precio* y *cantidad* deberían ser clases no primitivas *Cantidad* o *Moneda* porque se trata de cantidades en una unidad monetaria.
- El atributo de la *direccion* debe ser una clase no primitiva *Direccion* porque tiene secciones diferentes.

Las clases *ArticuloID*, *Direccion* y *Cantidad* son tipos de datos (no es significativa la identidad única de las instancias) pero merece la pena considerarlas como clases independientes debido a sus cualidades.

¿Dónde representamos las clases de tipos de datos?

¿Debería mostrarse la clase *ArticuloID* como una clase conceptual independiente en el modelo del dominio? Depende de lo que quiera resaltar en el diagrama. Puesto que *ArticuloID* es un *tipo de datos* (no es importante la identidad única de las instancias), se podría mostrar en el compartimento de los atributos del rectángulo de la clase, como se muestra en la Figura 12.4. Pero, puesto que es una clase no primitiva, con sus propios atributos y asociaciones, podría ser interesante mostrarla como una clase conceptual en su propio rectángulo. No existe una respuesta correcta; depende de cómo se esté utilizando el modelo del dominio como herramienta de comunicación, y la importancia de los conceptos en el dominio.

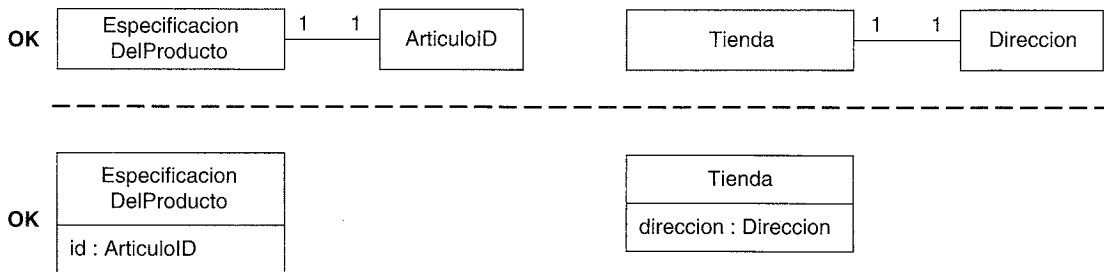


Figura 12.4. Si la clase del atributo es un tipo de datos, podría mostrarse en el rectángulo del atributo.

Un modelo del dominio es una herramienta de comunicación; las elecciones sobre lo que se muestra deben hacerse con esa consideración en mente.

12.5. Deslizarse al diseño: ningún atributo como clave ajena

No se deberían utilizar los atributos para relacionar las clases conceptuales en el modelo del dominio. La violación más típica de este principio es añadir un tipo de **atributo de clave ajena**, como se hace normalmente en el diseño de bases de datos relacionales, para asociar dos tipos. Por ejemplo, en la Figura 12.5, no es deseable el atributo *numeroRegistroActual* en la clases *Cajero* porque el propósito es relacionar el *Cajero* con un objeto *Registro*. La mejor manera de expresar que un *Cajero* utiliza un *Registro* es con una asociación, no con un atributo de clave ajena. Una vez más, relacione los tipos con una asociación, no con un atributo.

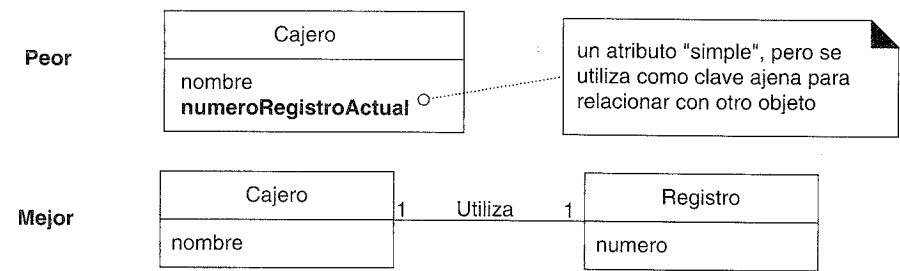


Figura 12.5. No utilice atributos como claves ajenas.

Hay muchas formas de relacionar objetos —siendo las claves ajenas una de ellas— y pospondremos el modo de implementar la relación hasta el diseño, para evitar el deslizamiento al diseño.

12.6. Modelado de cantidades y unidades de los atributos

La mayoría de las cantidades numéricas no deberían representarse simplemente como números. Considere el precio o la velocidad. Son cantidades con unidades asociadas, y es habitual que se necesite conocer la unidad y dar soporte a las conversiones. El software del PDV NuevaEra es para un mercado internacional y necesita soportar los precios en diferentes monedas. En el caso general, la solución consiste en representar la *Cantidad* como una clase conceptual aparte, con una *Unidad* asociada [Fowler96]. Puesto que las cantidades se consideran tipos de datos (no es importante la identidad única de las instancias), es aceptable recoger su representación en la sección de atributos del rectángulo de clase (ver Figura 12.6). También es común mostrar especializaciones de *Cantidad*. El *Dinero* es una clase de *Cantidad* cuyas unidades son monedas. El *Peso* es una cantidad cuyas unidades son kilogramos o libras.

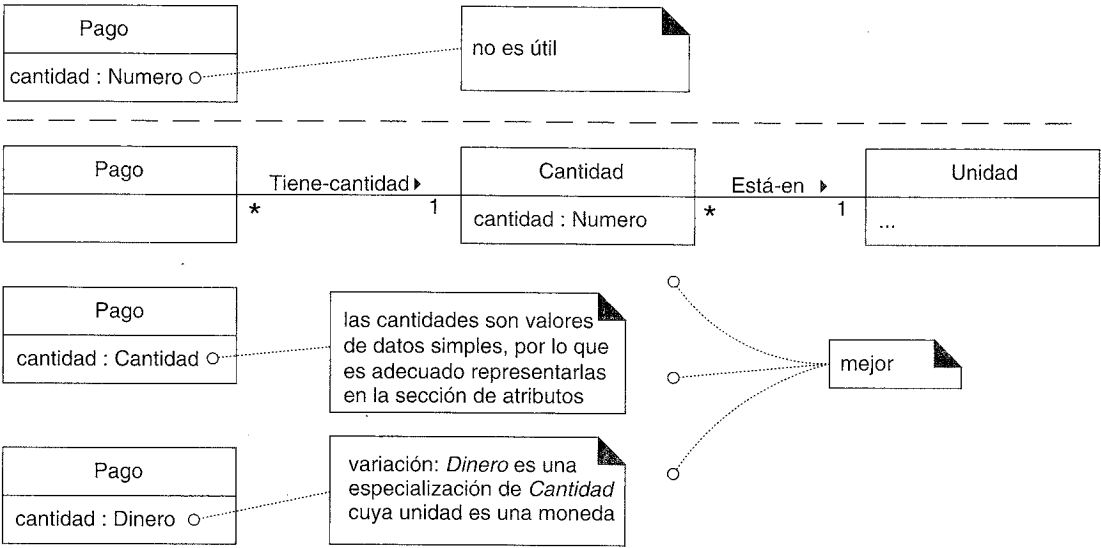


Figura 12.6. Modelado de cantidades.

12.7. Atributos en el Modelo del Dominio de NuevaEra

Los atributos elegidos reflejan los requisitos de esta iteración —los escenarios de *Procesar Venta* de esta iteración—.

<i>Pago</i>	<i>cantidad</i> : Se debe capturar una cantidad (también conocida como “cantidad entregada”) para determinar si se proporciona el pago suficiente y calcular el cambio.
<i>Especificacion DelProducto</i>	<i>descripcion</i> : Para mostrar la descripción en una pantalla o recibo. <i>id</i> : Para buscar una <i>EspecificacionDelProducto</i> , dado un artículoID introducido, es necesario relacionarlas con un <i>id</i> . <i>precio</i> : Para calcular el total de la venta y mostrar el precio de la línea de venta.
<i>Venta</i>	<i>fecha, hora</i> : Un recibo es un informe en papel de una venta. Normalmente muestra la fecha y la hora de la venta.
<i>LineaDeVenta</i>	<i>cantidad</i> : Para registrar la cantidad introducida, cuando hay más de un artículo en una línea de venta (por ejemplo, <i>cinco</i> paquetes de tofu).
<i>Tienda</i>	<i>direccion, nombre</i> : El recibo requiere el nombre y la dirección de la tienda.

12.8. Multiplicidad de la LíneaDeVenta al Artículo

Es posible que el cajero reciba un grupo de artículos iguales (por ejemplo, seis paquetes de tofu), introduzca el *articuloID* una vez, y después introduzca una cantidad (por

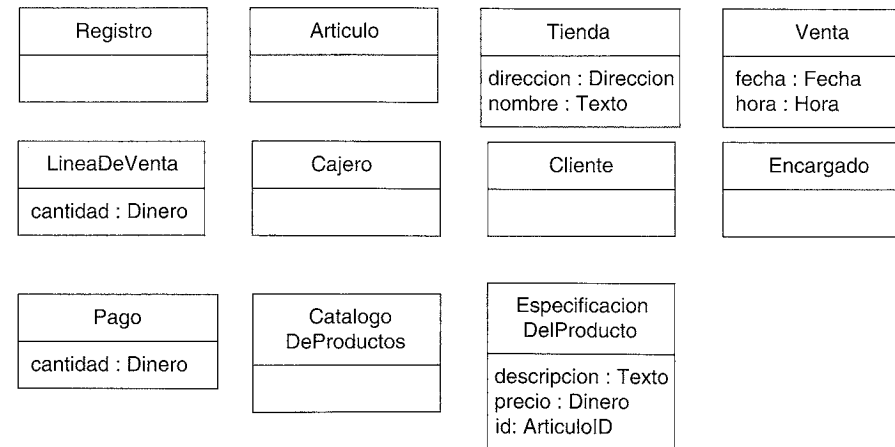


Figura 12.7. Modelo del dominio que muestra los atributos.

ejemplo, seis). En consecuencia, una *LineaDeVenta* individual se puede asociar con más de una instancia de un artículo.

La cantidad introducida por el cajero podría registrarse como un atributo de la *LineaDeVenta* (Figura 12.8). Sin embargo, la cantidad se puede calcular a partir del valor de la multiplicidad actual de la relación, de ahí que pudiera caracterizarse como un **atributo derivado** —uno que puede derivarse a partir de otra información—. En UML, un atributo derivado se indica con el símbolo “/”.

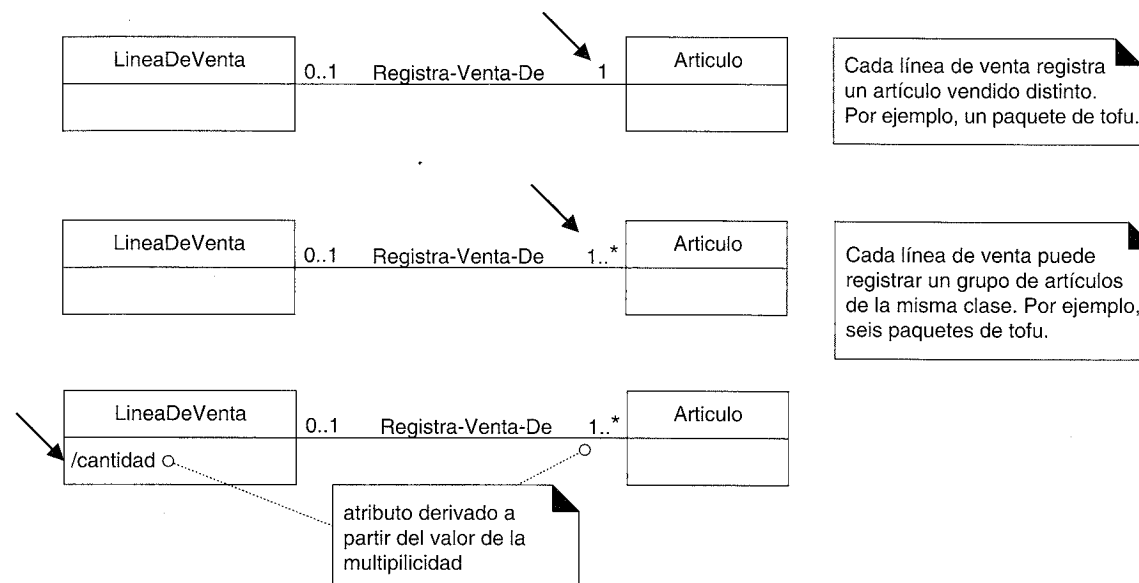


Figura 12.8. Registro de la cantidad de artículos vendidos en una línea de venta.

12.9. Conclusión del Modelo del Dominio

La combinación de las clases conceptuales, asociaciones y atributos, descubiertos en el estudio anterior, da lugar al modelo que se presenta en la Figura 12.9.

Se ha creado un modelo del dominio, relativamente útil, para el dominio de una aplicación de PDV. No existe un único modelo correcto. Todos los modelos son aproximaciones del dominio que estamos intentando entender. Un buen modelo del dominio captura las abstracciones y la información esenciales necesarias para entender el dominio en el contexto de los requisitos actuales, y ayuda a la gente a entender el dominio —sus conceptos, terminología y relaciones—.

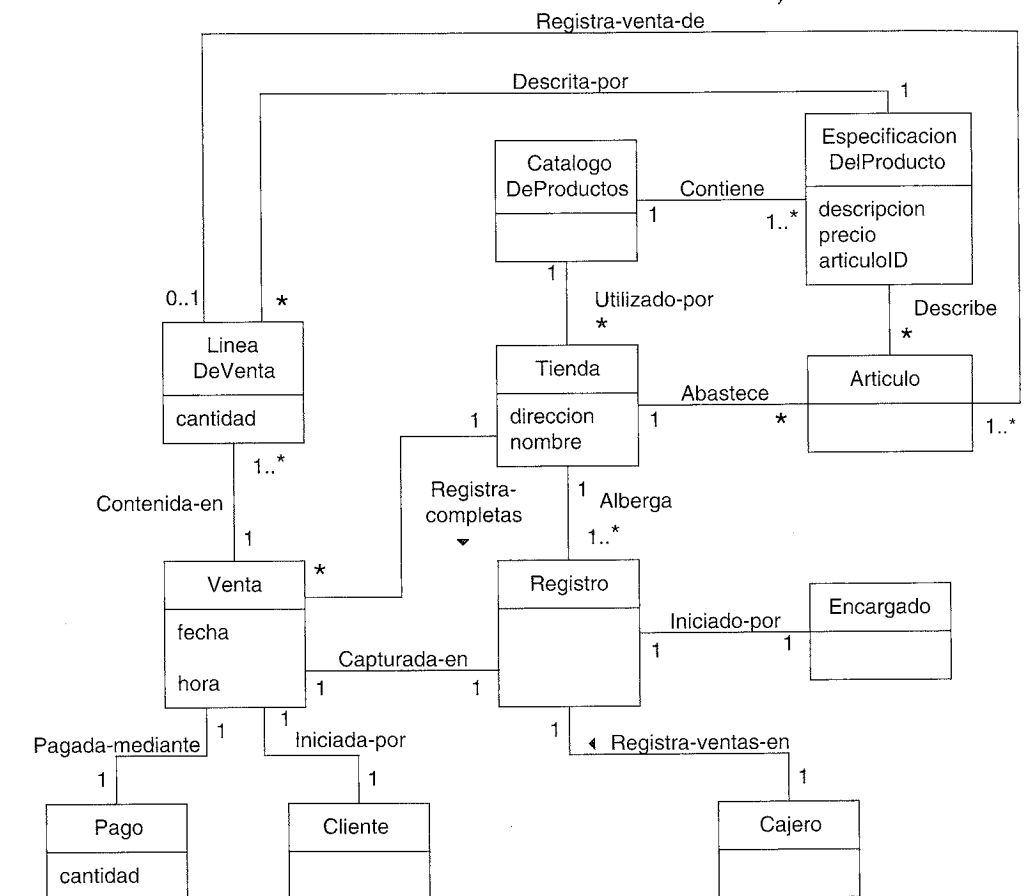


Figura 12.9. Un modelo del dominio parcial.

MODELO DE CASOS DE USO: AÑADIR DETALLES CON LOS CONTRATOS DE LAS OPERACIONES

Rápido, barato, bueno: elija dos cualquiera.

Anónimo

Objetivos

- Crear los contratos para las operaciones del sistema.
-

Introducción

Los contratos de las operaciones pueden ayudar a definir el comportamiento del sistema; describen el resultado de la ejecución de las operaciones del sistema en función de los cambios de estado de los objetos del dominio. Este capítulo explora su uso.

13.1. Contratos

Los casos de uso son el principal mecanismo del UP para describir el comportamiento del sistema y, normalmente es suficiente. Sin embargo, algunas veces se necesita una descripción más detallada del comportamiento del sistema. Los contratos describen el comportamiento detallado del sistema en función de los cambios de estado de los objetos del Modelo del Dominio, después de la ejecución de una operación del sistema.

Operaciones del sistema y la interfaz del sistema

Se pueden definir contratos para las **operaciones del sistema** —operaciones que el sistema, como una caja negra, ofrece en su interfaz pública para manejar los eventos del sistema entrantes—. Las operaciones del sistema se pueden identificar descubriendo estos eventos del sistema, como se muestra en la Figura 13.1.

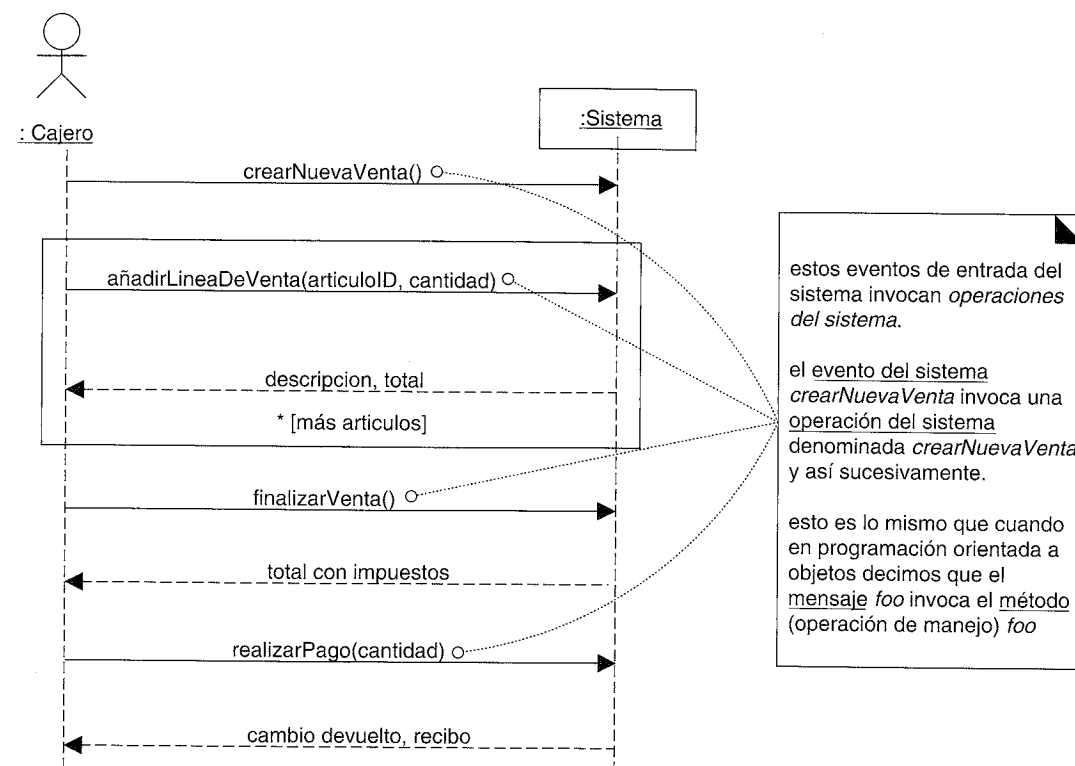


Figura 13.1. Las operaciones del sistema manejan los eventos de entrada del mismo.

El conjunto completo de operaciones del sistema, de todos los casos de uso, define la interfaz pública del sistema, viendo al sistema como un componente o clase individual. En UML, el sistema como un todo se puede representar mediante una clase.

13.2. Ejemplo de contrato: introducirArticulo

Antes de examinar las razones para la escritura de un contrato, merece la pena presentar un ejemplo. A continuación, se presenta un contrato para la operación del sistema *introducirArticulo*.

Contrato CO2: introducirArticulo

Operación:	introducirArticulo(articuloID:ArticuloID, cantidad:integer)
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de LineaDeVenta ldv (creación de instancias). - ldv se asoció con la Venta actual (formación de asociaciones). - ldv.cantidad pasó a ser cantidad (modificación de atributos). - ldv se asoció con una EspecificacionDelProducto, en base a la coincidencia del articuloID (formación de asociaciones).

13.3. Secciones del contrato

La descripción de cada una de las secciones del contrato se muestra en el siguiente esquema.

Operación:	Nombre de la operación y parámetros.
Referencias cruzadas:	(opcional) Casos de uso en los que pueden tener lugar esta operación.
Precondiciones:	<i>Suposiciones</i> relevantes sobre el estado del sistema o de los objetos del Modelo del Dominio, antes de la ejecución de la operación. No se comprobará en la lógica de esta operación, se asume que son verdad, y son suposiciones no triviales que el lector debe saber que se hicieron.
Postcondiciones:	- El estado de los objetos del Modelo del Dominio después de que se complete la operación. Se discute con detalle en la siguiente sección.

13.4. Postcondiciones

Nótese que cada una de las postcondiciones del ejemplo *introducirArticulo* incluía una categorización como *creación de instancias* o *formación de asociaciones*. He aquí un punto clave:

La postcondición describe cambios en el estado de los objetos del Modelo del Dominio. Los cambios de estado del Modelo del Dominio comprenden la creación de instancias, formación o rotura de asociaciones y cambio en los atributos.

Las postcondiciones no son acciones que se ejecutarán durante la operación; más bien, son declaraciones sobre los objetos del Modelo del Dominio que son verdad cuando la operación ha terminado —*después de que el humo se haya despejado*—.

En resumen, las postcondiciones se dividen en estas categorías:

- Creación y eliminación de instancias.
- Modificación de atributos.
- Formación y rotura de asociaciones (siendo precisos, *enlaces* UML).

Como ejemplo de postcondición que rompe una asociación, considere una operación que permite la eliminación de líneas de venta. La post-condición podría decir “Se rompió la asociación seleccionada de la *LineaDeVenta* con la *Venta*”. En otros dominios, cuando se cancela un préstamo o cuando alguien deja de ser socio de alguna organización, se rompen las asociaciones.

La postcondición de la eliminación de instancias es más rara, porque en el mundo real uno, normalmente, no se preocupa de forzar explícitamente la destrucción de una cosa. Sin embargo, como ejemplo: en muchos países, después de que una persona se

haya declarado en bancarrota y hayan pasado siete o diez años, se deben destruir todos los registros de su declaración de bancarrota, por ley. Nótese que esto es una perspectiva conceptual, no de implementación. Éstos no son declaraciones sobre liberar la memoria del ordenador ocupada por objetos software.

La cualidad importante es ser declarativo y enunciar con un estilo orientado al cambio en lugar de orientado a la acción, puesto que las postcondiciones son declaraciones sobre los estados o resultados, en lugar de una descripción de las acciones a ejecutar, o un diseño de una solución.

Las postcondiciones se relacionan con el Modelo del Dominio

Estas postcondiciones se expresan en el contexto de los objetos del Modelo del Dominio. ¿Qué instancias se pueden crear? —aquellas del Modelo del Dominio; ¿qué asociaciones se pueden formar?— las que se encuentran en el Modelo del Dominio; y así sucesivamente.

Una ventaja de las postcondiciones: detalle analítico

Expresados en un estilo declarativo de cambio de estado, los contratos son una herramienta excelente para el análisis de requisitos que describen los cambios de estado que requiere una operación del sistema (en función de los objetos del Modelo del Dominio) sin tener que describir cómo se van a llevar a cabo. En otras palabras, el diseño del software y la solución se puede diferir, y uno puede centrarse, analíticamente en qué debe suceder, en lugar de en cómo se va a realizar. Además, las postcondiciones soportan detalles de grano fino y una declaración más específica de cuál debe ser el resultado de la operación.

También es posible expresar este nivel de detalle en los casos de uso, pero normalmente no es deseable, puesto que entonces pasan a ser excesivamente elocuentes y detallados.

Considere la postcondición:

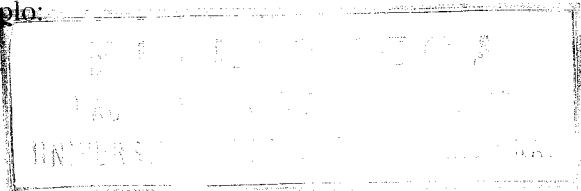
Postcondiciones:	- Se creó una instancia de LineaDeVenta ldv (creación de instancias). - ldv se asoció con la Venta actual (formación de asociaciones). - ldv.cantidad pasó a ser cantidad (modificación de atributos). - ldv se asoció con una EspecificacionDelProducto, en base a la coincidencia del articuloID (formación de asociaciones).
------------------	---

No se hace ningún comentario sobre el modo de crear una instancia de LineaDeVenta, o cómo se asocia con una Venta. Esto podría ser una declaración sobre escribir en folios y que se grapen, la utilización de la tecnología Java para crear objetos software y conectarlos, o la inserción de filas en una base de datos relacional.

El espíritu de las postcondiciones: el escenario y telón

Expresa las postcondiciones en pasado, para resaltar que son declaraciones sobre un cambio de estado en el pasado. Por ejemplo:

- (mejor) Se creó una LineaDeVenta
- (peor) Cree una LineaDeVenta.



- en lugar de
- Piense en las postcondiciones utilizando la siguiente imagen:
- El sistema y sus objetos se presentan en el escenario de un teatro.
1. Antes de la operación, tome una fotografía del escenario.
 2. Baje el telón y aplique la operación del sistema (*ruido metálico de fondo de martillos o campanas, gritos, chirridos...*).
 3. Suba el telón y tome una segunda fotografía.
 4. Compare las fotografías anterior y posterior, y exprese como postcondiciones el cambio en el estado del escenario (*Se creó una LineaDeVenta...*).

Si se utilizan contratos, ¿cómo de completas deben ser las postcondiciones?

Primero, los contratos podrían no ser necesarios. Esta cuestión se discute en una posterior sección. Pero, asumiendo que se desean algunos contratos, no es probable —o incluso necesario— que se genere un conjunto completo y detallado de postcondiciones para una operación del sistema, durante el trabajo de requisitos. Trate su creación como la mejor suposición inicial, entendiendo que los contratos no serán completos. Su temprana creación —incluso si es incompleta— es, ciertamente, mejor que diferir su estudio hasta el trabajo de diseño, cuando los desarrolladores deben preocuparse por el diseño de una solución, en lugar de investigar qué se debe hacer.

Algunos de los detalles finos —y quizás incluso los más importantes— se descubrirán durante el trabajo de diseño. Esto no es necesariamente una cosa mala; si el esfuerzo dedicado al análisis de requisitos es demasiado grande el rendimiento decrece. Naturalmente, tiene lugar algún descubrimiento durante el trabajo de diseño, que puede entonces documentar el trabajo de requisitos de una iteración posterior. Ésta es una de las ventajas del desarrollo iterativo: los descubrimientos que se generan en una iteración anterior pueden impulsar el estudio y el trabajo de análisis de la siguiente.

13.5. Discusión: postcondiciones de introducirArticulo

La siguiente sección analiza la motivación de las postcondiciones de la operación del sistema introducirArticulo.

Creación y eliminación de instancias

Después de introducir el *articuloID* y la *cantidad* de un artículo, ¿qué nuevo objeto debe haberse creado? Una *LineaDeVenta*. Por tanto:

- Se creó una instancia de *LineaDeVenta* *ldv* (creación de instancias).

Obsérvese el nombre de la instancia. Este nombre simplificará las referencias a la nueva instancia en otras sentencias de la post-condición.

Modificación de atributos

Después de que el cajero haya introducido el *articuloID* y la *cantidad*, ¿qué atributos de los objetos nuevos o de los ya existentes deberían haberse modificado? La *cantidad* de la *LineaDeVenta* debería haber pasado a ser igual que el parámetro *cantidad*. De ahí:

- *ldv.cantidad* pasó a ser *cantidad* (modificación de atributos).

Formación y rotura de asociaciones

Después de que el cajero haya introducido el *articuloID* y la *cantidad*, ¿qué asociaciones entre los objetos nuevos o los ya existentes deberían haberse formado o roto? La nueva *LineaDeVenta* debería haberse relacionado con su *Venta*, y su *EspecificacionDelProducto*. Así:

- *ldv* se asoció con la *Venta* actual (formación de asociaciones).
- *ldv* se asoció con una *EspecificacionDelProducto*, en base a la coincidencia del *articuloID* (formación de asociaciones).

Nótese la indicación informal de que se forma una relación con una *EspecificacionDelProducto* particular —aquella cuyo *articuloID* se corresponda con el parámetro—. Son posibles otros lenguajes más sofisticados y formales, como utilizar el Lenguaje de Restricciones de Objetos (OCL, *Object Constraint Language*). Recomendación: manténgalo simple.

13.6. La escritura de los contratos da lugar a actualizaciones en el Modelo del Dominio

Es normal, durante la creación de los contratos, descubrir la necesidad de registrar nuevas clases conceptuales, atributos o asociaciones en el Modelo del Dominio. No se limite a la definición anterior del Modelo del Dominio; enriquezca cuando haga nuevos descubrimientos mientras piensa en los contratos de las operaciones.

13.7. ¿Cuándo son útiles los contratos? ¿Contratos vs. casos de uso?

Los casos de uso son el principal repositorio de requisitos del proyecto. Podrían proporcionar la mayoría o todos los detalles necesarios para saber qué hacer en el diseño, en cuyo caso, los contratos no son útiles. Sin embargo, hay situaciones en las que los detalles y la complejidad de los cambios de estado requeridos, son difíciles de capturar en los casos de uso.

Por ejemplo, considere un sistema de reservas de vuelos y la operación del sistema *añadirNuevaReserva*. La complejidad es muy alta considerando todos los objetos del dominio que se deben cambiar, crear y asociar. Estos detalles de grano fino *se pueden* escribir en detalle en el caso de uso asociado a esta operación, pero dará lugar a un caso de uso extremadamente detallado (por ejemplo, anotando cada atributo que se debe cambiar en todos los objetos).

Obsérvese que el formato de la postcondición del contrato ofrece y promueve un lenguaje muy preciso, analítico y exigente que soporta una detallada minuciosidad.

Si, únicamente basándose en los casos de uso y mediante continuas colaboraciones (verbales) con un experto en la materia de estudio, los desarrolladores pueden entender cómodamente qué hacer, entonces evite la escritura de los contratos.

Sin embargo, en aquellas situaciones donde la complejidad es alta y añade valor la precisión detallada, los contratos son otra herramienta de requisitos.

Muy a menudo, los contratos no estarán muy justificados de manera que si un equipo está creando contratos para todas las operaciones del sistema de cada caso de uso, es una advertencia de que, o bien los casos de uso son algo deficientes, o no hay suficiente y continua colaboración o acceso a los expertos en la materia de estudio, o el equipo está haciendo demasiada documentación innecesaria.

El caso de estudio del PDV NuevaEra muestra más contratos de los que probablemente sean necesarios, por cuestiones pedagógicas. En la práctica, la mayoría de los detalles que recogen se pueden inferir de manera obvia a partir del texto de los casos de uso. Por otro lado, “obvio” es un concepto muy escurridizo.

13.8. Guías: contratos

Aplique el siguiente consejo para crear los contratos:

Para hacer contratos:

1. Identifique las operaciones del sistema a partir de los DSSs.
2. Construya un contrato para las operaciones del sistema complejas y quizás sutiles en sus resultados, o que no están claras en el caso de uso.
3. Para describir la postcondiciones utilice las siguientes categorías:
 - creación y eliminación de instancias
 - modificación de atributos
 - formación y rotura de asociaciones

Consejos acerca de la escritura de contratos

- Establezca las postcondiciones de forma declarativa, con una sentencia pasiva expresada en pasado para destacar que se trata de una declaración de un cambio de estado en lugar del diseño de la manera en la que se va a realizar. Por ejemplo:
 - (mejor) Se creó una *LineaDeVenta*
 - (peor) Cree una *LineaDeVenta*.
- Recuerde establecer una relación entre los objetos existentes o aquellos creados recientemente mediante la definición de la formación de asociaciones. Por ejemplo, no es suficiente que se cree una instancia de *LineaDeVenta* cuando tenga lugar la operación *introducirArticulo*. Después de que se complete la operación, también debería cumplirse que la nueva instancia creada se asoció con la *Venta*; de ahí que:
 - La *LineaDeVenta* se asoció con la *Venta* (formación de asociaciones).

El error más habitual en la creación de contratos

El problema más común es olvidarse de incluir la *formación de asociaciones*. En particular, cuando se crean nuevas instancias, es muy probable que se necesiten establecer asociaciones con varios objetos. ¡No lo olvide!

13.9. Ejemplo del PDV NuevaEra: contratos

Operaciones del sistema de Procesar Venta

Contrato CO1: crearNuevaVenta

Operación:	crearNuevaVenta()
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Ninguna
Postcondiciones:	- Se creó una instancia de Venta v (creación de instancias). - v se asoció con el Registro (formación de asociaciones). - Se inicializaron los atributos de v.

Obsérvese la descripción vaga de la última post-condición. Si es suficiente, está bien.

En un proyecto, todas estas postcondiciones particulares son tan obvias a partir del caso de uso que, probablemente, no se debería escribir el contrato de *crearNuevaVenta*.

Recuerde uno de los principios que guían un buen proceso y el UP: Manténgalo tan ligero como sea posible, y evite todos los artefactos a menos que realmente añadan valor.

Contrato CO2: introducirArticulo

Operación:	introducirArticulo(articuloID:ArticuloID, cantidad:integer)
Referencias cruzadas:	Caso de Uso: Procesar Venta

Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de LineaDeVenta ldv (creación de instancias). - ldv se asoció con la Venta actual (formación de asociaciones). - ldv.cantidad pasó a ser cantidad (modificación de atributos). - ldv se asoció con una EspecificacionDelProducto, en base a la coincidencia del articuloID (formación de asociaciones).

Contrato CO3: finalizarVenta

Operación:	finalizarVenta()
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Venta.esCompleta pasó a ser verdad (modificación de atributos).

Contrato CO4: realizarPago

Operación:	realizarPago(cantidad: Dinero)
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de Pago p (creación de instancias). - p.cantidadEntregada pasó a ser cantidad (modificación de atributos). - p se asoció con la Venta actual (formación de asociaciones). - La Venta actual se asoció con la Tienda (formación de asociaciones); (para añadirlo al registro histórico de las ventas completadas).

13.10. Cambios en el Modelo del Dominio

Hay un dato sugerido por estos contratos que todavía no está representado en el modelo del dominio: la terminación de la entrada del artículo en la venta. La especificación de *finalizarVenta* lo modifica, y probablemente es una buena idea comprobarlo después, durante el trabajo de diseño, en la operación *realizarPago*, para rechazar los pagos hasta que se complete una venta.

Una manera de representar esta información es con un atributo *esCompleta* en la *Venta*, de tipo de dato booleano:

Venta
esCompleta: Boolean
fecha
hora

Existen alternativas, que se tienen en cuenta, especialmente, durante el trabajo de diseño. Una técnica se denomina el **patrón de Estado**, que se estudiará en el Capítulo 34. Otra es el uso de objetos “sesión” que siguen la pista del estado de una sesión y rechazan operaciones impropcedentes; esto también se estudiará más tarde.

13.11. Contratos, operaciones y UML

Contratos en UML: especificación de operaciones

UML define las **operaciones** formalmente. Citando textualmente:

Una operación es una especificación de una transformación o consulta que se puede invocar para que la ejecute un objeto. [RJB99]

Por ejemplo, los elementos de una interfaz son operaciones, en términos de UML. Una operación es una abstracción, no una implementación. Por el contrario, un **método** (en UML) es una implementación de una operación.

Una operación UML tiene una **signatura** (nombre y parámetros), y también una **especificación de operación**, que describe los efectos producidos por la ejecución de la operación; esto es, la postcondición. El formato de la especificación de operación en UML es flexible, y no tiene que ser el formato de contrato que se muestra en este capítulo. Sin embargo, los documentos de UML proporcionan como ejemplos el estilo de contratos con pre- y postcondiciones, ya que éste es el enfoque más conocido para las especificaciones formales de las operaciones.

Resumiendo: UML define especificaciones de operaciones, que se pueden especificar con el estilo de los contratos con pre- y postcondiciones. Nótese que, como se subraya en este capítulo, una especificación de operación UML podría *no* mostrar un algoritmo o solución, sino únicamente los cambios de estado o efectos de las operaciones.

Además de utilizar los contratos para especificar las operaciones públicas del *Sistema* completo (operaciones del sistema), los contratos se pueden aplicar a las operaciones de cualquier nivel de granularidad: las operaciones públicas (o interfaz) de un subsistema, una clase abstracta, etcétera. Las operaciones presentadas en este capítulo pertenecen a la clase *Sistema*. En UML, las operaciones pertenecen a las clases. Más aún, en UML, los “subsistemas” se modelan como clases (y simultáneamente también como paquetes). En UML, el “sistema” global es el subsistema de más alto nivel, y se modela como una clase denominada *Sistema* (en realidad, cualquier nombre es legal) con operaciones y especificaciones públicas.

Contratos de las operaciones expresados con OCL

Existe un lenguaje formal asociado con UML denominado Lenguaje de Restricciones de Objetos (**OCL**, *Object Constraint Language*) [WK99], que se puede utilizar para expresar las restricciones en los modelos. OCL podría utilizarse en lugar del lenguaje natural informal que se utiliza en este capítulo; UML permite cualquier formato para una especificación de operación.

Sugerencia

A menos que exista una razón práctica apremiante que requiera que la gente aprenda y utilice OCL, mantenga las cosas simples y utilice el lenguaje natural.

OCL define un formato oficial para la especificación de las pre- y postcondiciones de las operaciones, como se muestra en este fragmento:

```
Sistema::crearNuevaVenta( )
pre: <sentencia en OCL>
post: ...
```

Detalles adicionales sobre OCL están fuera del alcance de este libro.

Contratos en el Diseño por Contrato

La forma de los contratos con pre- y postcondiciones utilizados para la especificación de las operaciones en UML, se lleva impulsando durante muchos años por Bertrand Meyer, formalizado en una técnica de diseño denominada **Diseño por Contrato** [Meyer97 (primera ed. 1989)], aunque su origen procede de un trabajo anterior en los años sesenta sobre los lenguajes de especificación formal. En el Diseño por Contrato, también se escriben los contratos para las operaciones de las clases de grano fino, no sólo para las operaciones públicas de los sistemas y subsistemas.

Además, el Diseño por Contrato fomenta la inclusión de una sección *invariante*, como es habitual en especificaciones de contrato completas. Los invariantes definen las cosas que no deben cambiar de estado antes y después de que se ejecute una operación. Los invariantes no se han utilizado en este capítulo por simplicidad.

Soporte de los lenguajes de programación para los contratos

Algunos lenguajes, como Eiffel, incluyen soporte a nivel de expresiones del lenguaje para los invariantes, las pre- y postcondiciones. Existen pre-procesadores en Java que proporcionan un soporte parecido.

13.12. Contratos de las operaciones en el UP

Un contrato con pre- y postcondiciones es un estilo bien conocido para especificar una operación en UML. En UML, las operaciones existen a muchos niveles, desde el *Sistema* hasta las clases de grano fino, como *Venta*. Los contratos de especificación de operaciones para el nivel del *Sistema* forman parte del Modelo de Casos de Uso, aunque no se pusieron de relieve en la documentación original del RUP o el UP; su inclusión en este modelo se verificó con los autores del RUP¹.

Fases

- Inicio:** Los contratos no se justifican durante la fase de inicio, son demasiado detallados.
- Elaboración:** Si es que se utilizan, los contratos se escribirán durante la elaboración, cuando se escriben la mayoría de los casos de uso. Escriba solamente los contratos para las operaciones del sistema más complejas y sutiles.

Relaciones entre los artefactos

En las Figuras 13.2 y 13.3 se muestran las relaciones entre los contratos y otros artefactos, con diferentes niveles de detalle.

¹ Comunicación privada.

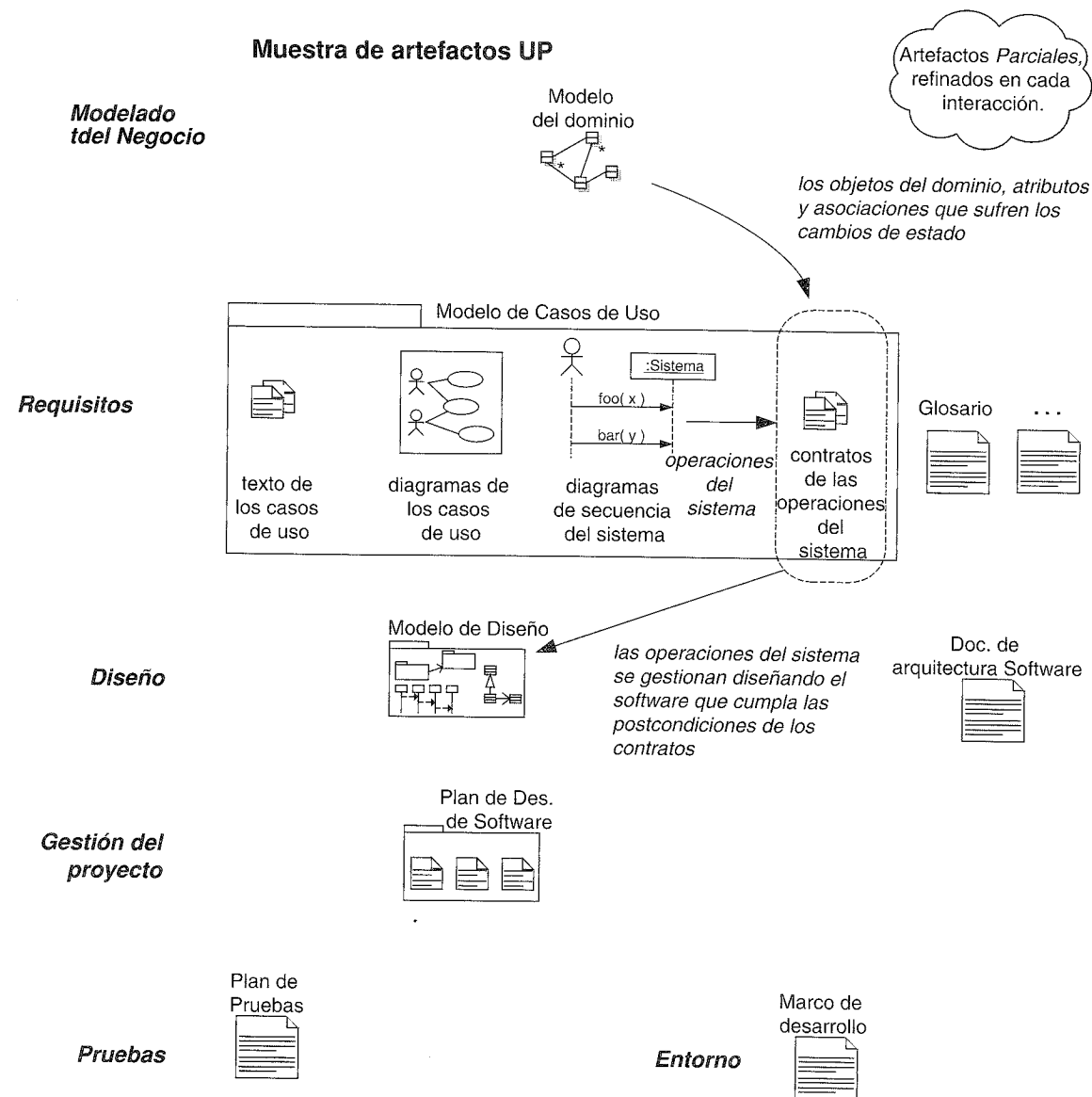


Figura 13.2. Muestra de la influencia entre los artefactos UP.

13.13. Lecturas adicionales

Los contratos de las operaciones proceden del área de las especificaciones formales, y se llevan utilizando y refinando desde los años sesenta, como en el Método de Desarrollo Viena (VDM, *Vienna Development Method*) [BJ78]; existe literatura abundante sobre VDM y otros lenguajes de especificación formal.

Bertrand Meyer contribuyó a un reconocimiento mucho más amplio de las especificaciones formales y los contratos con la inclusión de las pre- y postcondiciones en el lenguaje Eiffel; su *Construcción de Software Orientado a Objetos* proporciona los detalles. Es el creador del **Diseño por Contrato**.

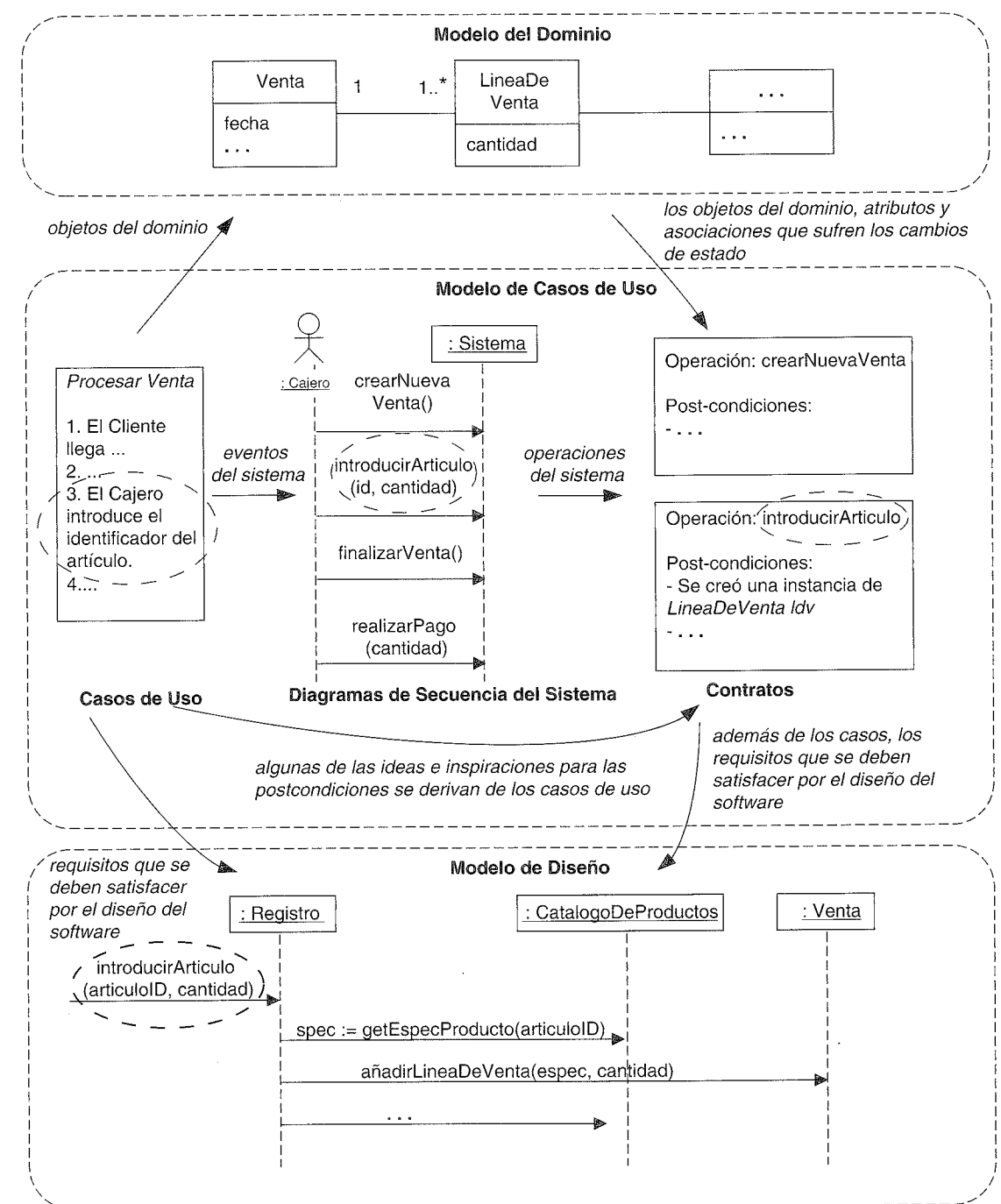


Figura 13.3. Relación del contrato con otros artefactos.

En UML, los contratos de las operaciones también pueden especificarse de manera más rigurosa en el Lenguaje de Restricciones de Objetos (OCL), para lo que se requiere leer *The Object Constraint Language: Precise Modeling with UML*, de Warmer y Kleppe.

DE LOS REQUISITOS AL DISEÑO EN ESTA ITERACIÓN

Hardware, n.: Las partes de un ordenador a las que se pueden dar patadas.

Anónimo

Objetivos

- Motivar la transición a las actividades de diseño.
 - Contrastar la importancia de las técnicas de diseño de objetos frente al conocimiento de la notación UML.
-

Introducción

Hasta el momento, el caso de estudio ha hecho hincapié en el estudio de los requisitos, conceptos y operaciones relacionadas con un sistema. Siguiendo las guías del UP, quizás se investigaron el 10% de los requisitos en la fase de inicio, y se comenzó un estudio ligeramente más profundo en esta primera iteración de la elaboración. Los capítulos siguientes cambian el énfasis hacia el diseño de una solución para esta iteración en función de objetos software que colaboran.

14.1. Iterativamente hacer lo correcto, y hacerlo correcto

Los requisitos y el análisis orientado a objetos se han centrado en aprender a *hacer lo correcto*; es decir, en entender algunos de los objetivos importantes del PDV NuevaEra, y las reglas y restricciones relacionadas. Por el contrario, el siguiente trabajo de diseño pondrá de relieve *hacerlo correcto*; esto es, diseñar con destreza una solución que satisfaga los requisitos de esta iteración.

En el desarrollo iterativo, en cada iteración tendrá lugar una transición desde un enfoque centrado en los requisitos a un enfoque centrado en el diseño y la implementación.

Además, es normal y saludable descubrir y cambiar algunos requisitos de las *primeras* iteraciones durante el trabajo de diseño e implementación. Estos descubrimientos clarificarán los objetivos del trabajo de diseño de esta iteración, y refinarán la comprensión de los requisitos para las iteraciones futuras. A lo largo de estas primeras iteraciones de elaboración el descubrimiento de los requisitos se estabilizará, de manera que al final de la elaboración quizás se definan con detalle y de manera fiable el 80% de los requisitos.

14.2. ¿No lleva eso semanas en hacerse? No, no exactamente

Después de muchos capítulos de discusión detallada, seguramente, debe parecer que el modelado anterior debe llevar semanas de esfuerzo. No es así. Cuando uno se encuentra cómodo con las técnicas de escritura de casos de uso, modelado del dominio, etcétera, el tiempo necesario para llevar a cabo el modelado real que hemos estudiado hasta el momento, de modo realista, será sólo de unos *pocos* días.

Sin embargo, eso no significa que sólo hayan pasado unos días desde el comienzo del proyecto. Muchas otras actividades, como programación de pruebas de conceptos, búsqueda de recursos (personas, software,...), planificación, acondicionamiento del entorno, etcétera, podrían consumir unas pocas semanas de preparación.

14.3. Pasar al diseño de objetos

Durante el diseño de objetos, se desarrolla una solución lógica basada en el paradigma orientado a objetos. Lo esencial de esta solución es la creación de los **diagramas de interacción**, que representan el modo en el que los objetos colaboran para satisfacer los requisitos.

Después de —o en paralelo con— la elaboración de los diagramas de interacción, se pueden representar los **diagramas de clases** (del diseño). Éstos resumen la definición de las clases software (e interfaces) que se van a implementar en el software.

Por lo que se refiere al UP, estos artefactos forman parte del **Modelo de Diseño**.

En la práctica, la creación de los diagramas de interacción y clases tiene lugar en paralelo y con sinergia entre ellos, aunque se introducen de manera lineal en este caso de estudio, por simplicidad y claridad.

La importancia de las técnicas del diseño de objetos vs. las técnicas de la notación UML

Los capítulos siguientes estudian la creación de estos artefactos, o de manera más precisa, las técnicas del diseño de objetos que subyacen a sus creaciones. Lo que es importante es saber cómo pensar y diseñar con objetos, que es muy diferente y una capa-

cidad más importante, que conocer la notación de los diagramas de UML. Al mismo tiempo, un lenguaje visual estándar es importante y, por tanto, se presenta la notación UML necesaria para dar soporte al trabajo de diseño.

De los dos artefactos que se estudiarán, los diagramas de interacción son los más importantes —desde el punto de vista del desarrollo de un buen diseño— y requiere el mayor grado de esfuerzo creativo. La creación de los diagramas de interacción requiere la aplicación de los principios para la asignación de **responsabilidades** y el uso de los **principios y patrones de diseño**. Por tanto, el énfasis de los siguientes capítulos se pone en estos principios y patrones del diseño de objetos.

Las técnicas del diseño de objetos vs. las técnicas de la notación UML

La representación de los diagramas de interacción UML es un reflejo de la toma de decisiones sobre el diseño de objetos.

Las técnicas del diseño de objetos son lo que realmente importan, en lugar de saber cómo dibujar los diagramas UML.

El diseño de objetos básico requiere que se tenga conocimiento sobre:

- Los principios de asignación de responsabilidades.
- Patrones de diseño.

NOTACIÓN DE LOS DIAGRAMAS DE INTERACCIÓN

Los gatos son más listos que los perros. No puedes conseguir que ocho gatos empujen un trineo por la nieve.

Jeff Valdez

Objetivos

- Leer la notación de los diagramas de interacción (secuencia y colaboración) UML básicos.
-

Introducción

Los siguientes capítulos estudian el diseño de objetos. El lenguaje utilizado para ilustrar los diseños es, principalmente, los diagramas de interacción. Por tanto, es aconsejable, por lo menos, examinar superficialmente los ejemplos de este capítulo y familiarizarse con la notación antes de avanzar.

UML incluye los **diagramas de interacción** para ilustrar el modo en el que los objetos interaccionan por medio de mensajes. Este capítulo introduce la notación, mientras que los capítulos siguientes se centran en su uso en el contexto del aprendizaje y realización del diseño de objetos para el caso de estudio del PDV NuevaEra.

Lea los siguientes capítulos para las guías de diseño

Este capítulo introduce la notación. Para crear objetos bien diseñados, también se deben entender los principios de diseño. Después de familiarizares algo con la notación de los diagramas de interacción, es importante estudiar los siguientes capítulos sobre estos principios y cómo aplicarlos mientras se dibujan los diagramas de interacción.

15.1. Diagramas de secuencia y colaboración

EL término *diagrama de interacción* es una generalización de dos tipos de diagramas UML más especializados; ambos pueden utilizarse para representar de forma similar interacciones de mensajes:

- Diagramas de colaboración.
- Diagramas de secuencia.

A lo largo del libro, se utilizarán ambos tipos, para remarcar la flexibilidad en la elección.

Los **diagramas de colaboración** ilustran las interacciones entre objetos en un formato de grafo o red, en el cual los objetos se pueden colocar en cualquier lugar del diagrama, como se muestra en la Figura 15.1.

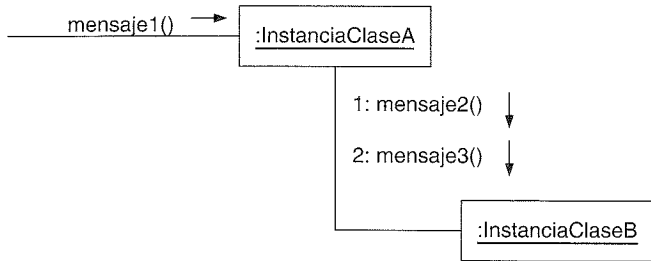


Figura 15.1. Diagrama de colaboración.

Los **diagramas de secuencia** ilustran las interacciones en un tipo de formato con el aspecto de una valla, en el que cada objeto nuevo se añade a la derecha, como se muestra en la Figura 15.2.

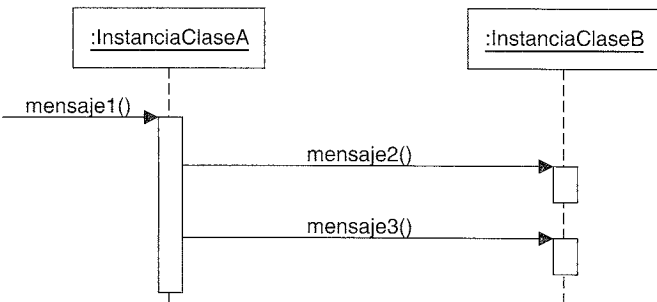


Figura 15.2. Diagrama de secuencia.

Cada tipo tiene puntos fuertes y débiles. Cuando se dibujan los diagramas para publicarlos en páginas estrechas, los diagramas de colaboración tienen la ventaja de permitir la expansión vertical para los nuevos objetos; los objetos adicionales en un diagrama de secuencia deben extenderse hacia la derecha, lo que supone una limitación. Por otro lado, los ejemplos de diagramas de colaboración dificultan que se vea fácilmente la secuencia de mensajes.

NOTACIÓN DE LOS DIAGRAMAS DE INTERACCIÓN A187

LA MAYORÍA PREFIEREN LOS DIAGRAMAS DE SECUENCIA CUANDO UTILIZAN UNA HERRAMIENTA CASE PARA HACER INGENIERÍA INVERSA DEL CÓDIGO FUENTE. LOS DIAGRAMAS DE INTERACCIÓN, PUNTO QUE MUESTRAN CLARAMENTE LA SECUENCIA DE MENSAJES.

Tipo	Puntos fuertes	Puntos débiles
secuencia	muestra claramente la secuencia u ordenación en el tiempo de los mensajes notación simple	fuerza a extender por la derecha cuando se añaden nuevos objetos; consume espacio horizontal
colaboración	economiza espacio, flexibilidad al añadir nuevos objetos en dos dimensiones es mejor para ilustrar bifurcaciones complejas, iteraciones y comportamiento concurrente	difícil ver la secuencia de mensajes notación más compleja

15.2. Ejemplo de diagrama de colaboración: realizarPago

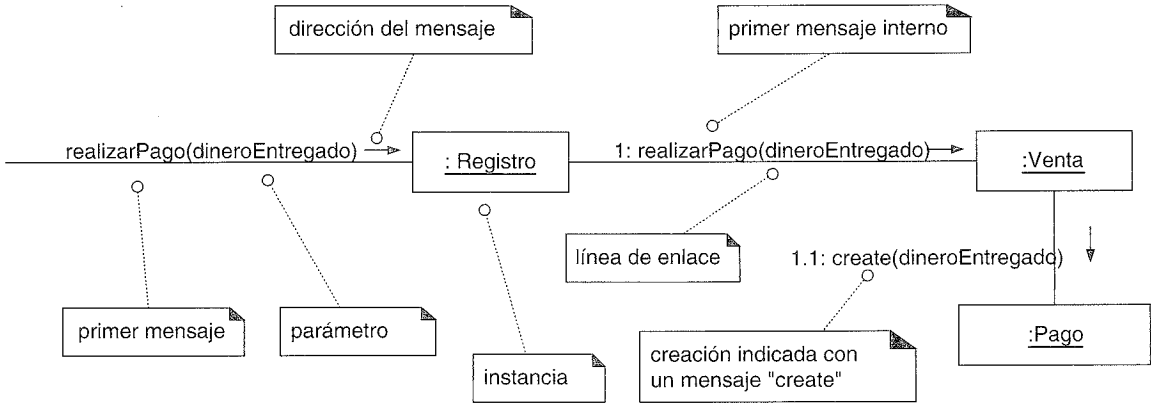


Figura 15.3. Diagrama de colaboración.

El diagrama de colaboración que se muestra en la Figura 15.3 se lee como sigue:

1. Se envía el mensaje *realizarPago* a una instancia de *Registro*. No se identifica al emisor.
2. La instancia de *Registro* envía el mensaje *realizarPago* a una instancia de *Venta*.
3. La instancia de *Venta* crea una instancia de *Pago*.

15.3. Ejemplo de diagrama de secuencia: realizarPago

El objetivo del diagrama de secuencia que se muestra en la Figura 15.4 es el mismo que el del anterior diagrama de colaboración.

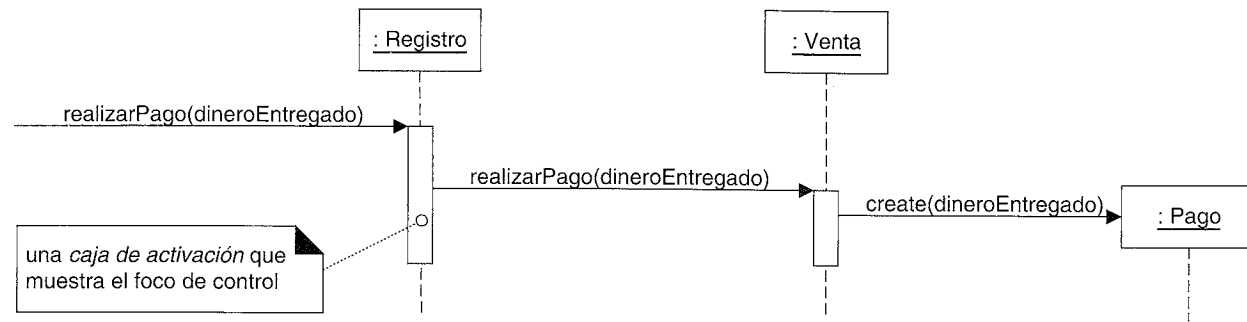


Figura 15.4. Diagrama de secuencia.

15.4. Los diagramas de interacción son importantes

Un problema típico en los proyectos de tecnología de objetos es que no aprecian el valor de llevar a cabo el diseño de objetos mediante el uso de los diagramas de interacción. Un problema relacionado es hacerlos de una manera vaga, como mostrando mensajes hacia objetos que realmente necesitan mucha elaboración adicional; por ejemplo, mostrando el mensaje *ejecutarSimulacion* a algún objeto *Simulacion*, pero sin continuar con el diseño más detallado, como pensando que en virtud de un mensaje con un buen nombre el diseño se completa mágicamente.

Se debería dedicar un tiempo y esfuerzo no trivial a la creación de diagramas de interacción, como reflejo de que se ha estudiado cuidadosamente los detalles del diseño de objetos. Por ejemplo, si la duración de la iteración es de dos semanas, quizás medio día o uno entero, cerca del comienzo de la iteración, se debería dedicar a la creación de los diagramas de interacción (y en paralelo, los diagramas de clases), antes de pasar a la programación. Sí, es cierto, el diseño que se representa en los diagramas será imperfecto y especulativo, y se modificará durante la programación, pero proporcionará un punto de partida serio, consistente y común, que sirve de inspiración durante la programación.

Sugerencia

Cree los diagramas de interacción por parejas, no solo. El diseño colaborando con otro será mejor, y los compañeros aprenderán rápidamente los unos de los otros.

Nótese que es principalmente durante esta etapa donde se requiere la aplicación de las técnicas de diseño, en términos de patrones, estilos y principios. La creación de los casos de uso, modelos del dominio, y otros artefactos, es más sencilla que la asignación de responsabilidades y la creación de diagramas de interacción bien diseñados. Esto es debido a que existen un gran número de principios de diseño sutiles y “grados de libertad” que subyacen a un diagrama de interacción bien diseñado, que a la mayoría de los otros artefactos de A/DOO.

La realización de los diagramas de interacción (en otras palabras, decidir sobre los detalles del diseño de objetos) es una etapa muy creativa del A/DOO. Se pueden aplicar patrones codificados, principios y estilos para mejorar la calidad de sus diseños.

Los principios de diseño que se necesitan para la construcción con éxito de los diagramas de interacción *pueden* codificarse, explicarse y aplicarse de forma sistemática. Este enfoque para entender y utilizar los principios de diseño se basa en los **patrones** —guías y principios estructurados—. Por tanto, después de introducir la sintaxis de los diagramas de interacción, volveremos la atención (en los siguientes capítulos) hacia los patrones de diseño y su aplicación a los diagramas de interacción.

15.5. Notación general de los diagramas de interacción

Representación de clases e instancias

UML ha adoptado un enfoque simple y consistente para representar las **instancias** frente a los clasificadores (ver Figura 15.5):

- Para cualquier tipo de elemento UML (clase, actor,...), una instancia utiliza el mismo símbolo gráfico que el tipo, pero con la cadena de texto que lo designa subrayada.

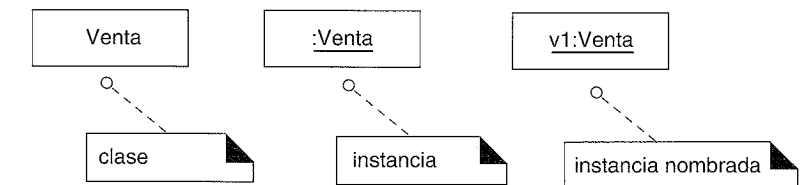


Figura 15.5. Clases e instancias.

En consecuencia, para mostrar una instancia de una clase en un diagrama de interacción, se utiliza el símbolo gráfico para una clase, esto es el rectángulo, pero con el nombre subrayado.

Se puede utilizar un nombre para identificar de manera única la instancia. Si no se utiliza, obsérvese que los “:” preceden al nombre de la clase.

Sintaxis de la expresión de mensaje básica

UML cuenta con una sintaxis básica para las expresiones de los mensajes:

```
return := mensaje(parametro: tipoParametro): tipoRetorno
```

Podría excluirse la información del tipo si es obvia o no es importante. Por ejemplo:

```
espec := getEspecProducto(id)
espec := getEspecProducto(id:ArticuloID)
espec := getEspecProducto(id:ArticuloID):EspecificacionDelProducto
```

15.6. Notación básica de los diagramas de colaboración

Enlaces

Un **enlace** es un camino de conexión entre dos objetos; indica que es posible alguna forma de navegación y visibilidad entre los objetos (ver Figura 15.6). De manera más formal, un enlace es una instancia de una asociación. Por ejemplo, existe un enlace —o camino de navegación— desde un *Registro* a una *Venta*, a lo largo del que pueden fluir los mensajes, como el mensaje *realizarPago*.

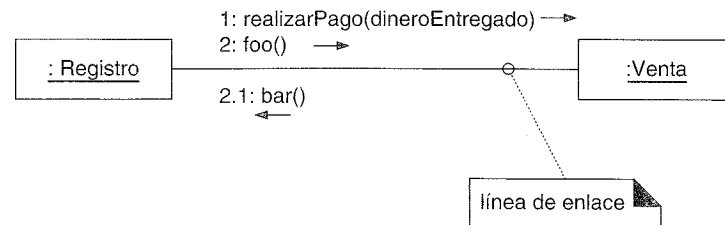


Figura 15.6. Líneas de enlaces.

Obsérvese que, a lo largo del mismo enlace, pueden fluir múltiples mensajes, y mensajes en ambas direcciones.

Mensajes

Cada mensaje entre objetos se representa con una expresión de mensaje y una pequeña flecha que indica la dirección del mensaje. Podrían fluir muchos mensajes a lo largo de este enlace (Figura 15.7). Se añade un número de secuencia para mostrar el orden secuencial de los mensajes en el hilo de control actual.

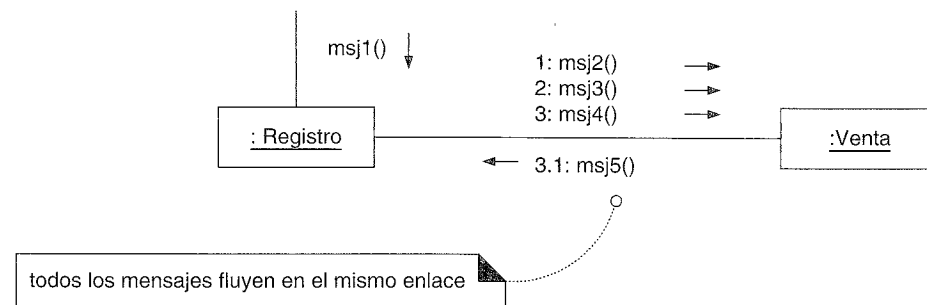


Figura 15.7. Mensajes.

Mensajes a “self” o “this”

Se puede enviar un mensaje desde un objeto a él mismo (Figura 15.8). Esto se representa mediante un enlace a él mismo, con mensajes que fluyen a lo largo del enlace.

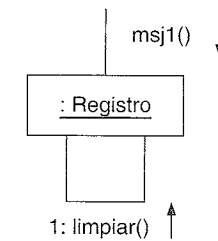


Figura 15.8. Mensajes a “this”.

Creación de instancias

Cualquier mensaje se puede utilizar para crear una instancia, pero en UML existe el convenio de utilizar para este fin el mensaje denominado *create*. Si se utiliza otro nombre de mensaje (quizás menos obvio), se podría añadir al mensaje una característica especial llamada estereotipo UML, como «*create*».

El mensaje *create* podría incluir parámetros, que indican el paso de valores iniciales. Esto indica, por ejemplo, la llamada a un constructor con parámetros en Java.

Además, podría añadirse opcionalmente la propiedad UML {*new*} a la caja de la instancia para resaltar la creación.

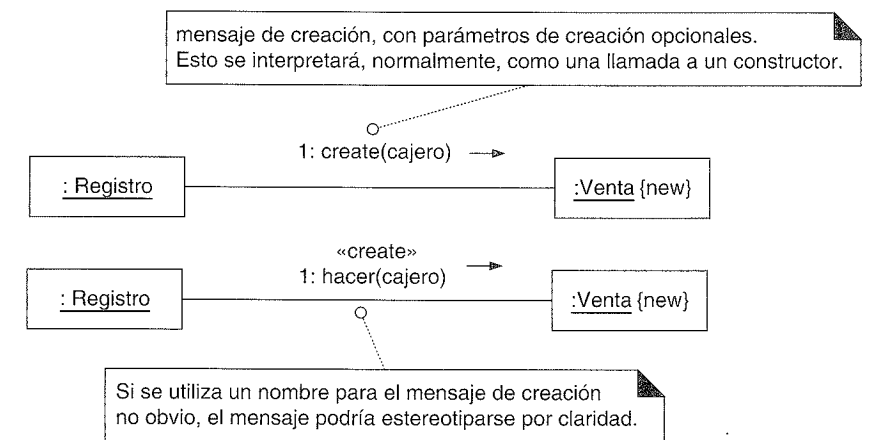


Figura 15.9. Creación de instancias.

Secuencia de números de mensaje

El orden de los mensajes se representa mediante **números de secuencia**, como se muestra en la Figura 15.10. El esquema de numeración es:

1. No se numera el primer mensaje. Por tanto, el *msj1()* no se numera.
2. El orden y anidamiento de los siguientes mensajes se muestran con el esquema de numeración válido en el que los mensajes anidados tienen un número adjunto. El anidamiento se denota anteponiendo el número del mensaje entrante al número del mensaje saliente.

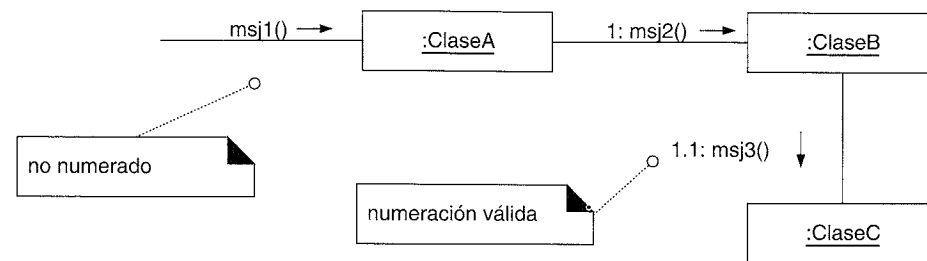


Figura 15.10. Secuencia de numeración.

En la Figura 15.11 se muestra un caso más complejo.

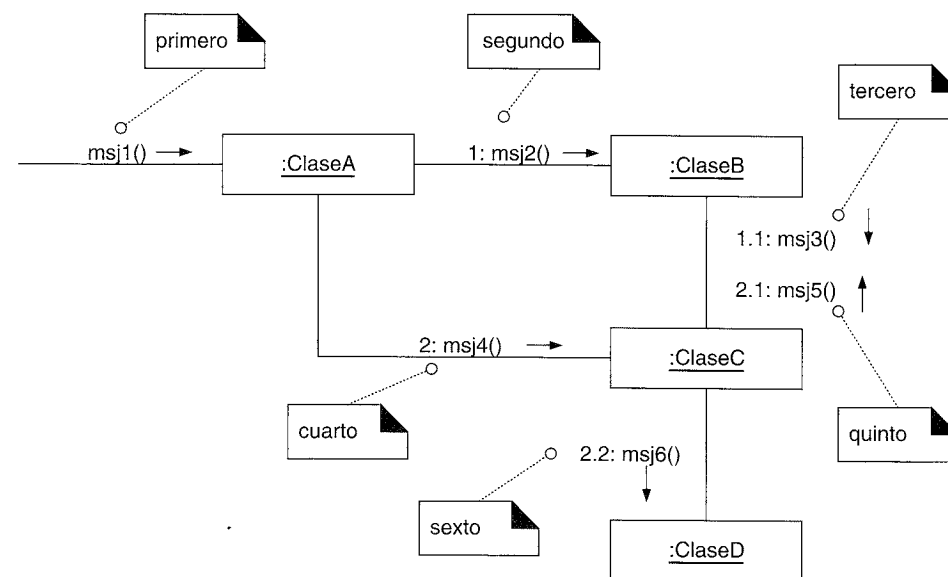


Figura 15.11. Secuencia de numeración compleja.

Mensajes condicionales

Un mensaje condicional (ver Figura 15.12) se muestra con una cláusula condicional, similar a una cláusula de iteración, entre corchetes, a continuación del número de secuencia. El mensaje sólo se envía si la evaluación de la cláusula es *verdad*.

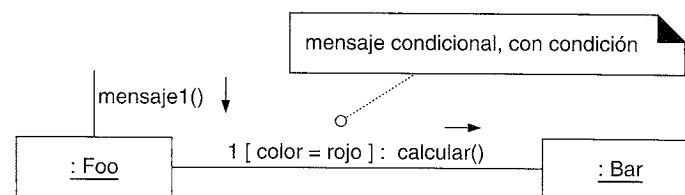


Figura 15.12. Mensaje condicional.

Caminos condicionales mutuamente exclusivos

El ejemplo de la Figura 15.13 ilustra los números de secuencia con caminos condicionales mutuamente exclusivos.

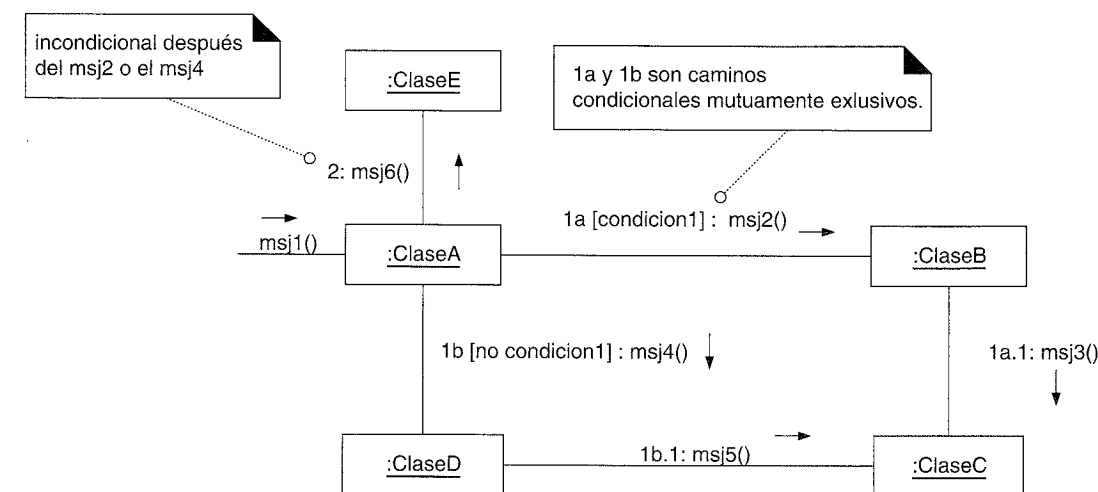


Figura 15.13. Mensajes mutuamente exclusivos.

En este caso es necesario modificar las expresiones de la secuencia con una letra de camino condicional. La primera letra que se utiliza es la **a** por convenio. La Figura 15.13 establece que se podría ejecutar o *1a* o *1b* después de *msj1()*. Ambos tienen el número de secuencia 1 puesto que cualquiera de ellos podría ser el primer mensaje interno.

Nótese que los siguientes mensajes anidados todavía se anteponen de manera consistente con su secuencia de mensaje exterior. De este modo *1b.1* es el mensaje anidado de *1b*.

Iteración o bucle

La notación de las iteraciones se muestra en la Figura 15.14. Si los detalles de la cláusula de iteración no son importantes para el modelador, se puede utilizar simplemente un “*”.

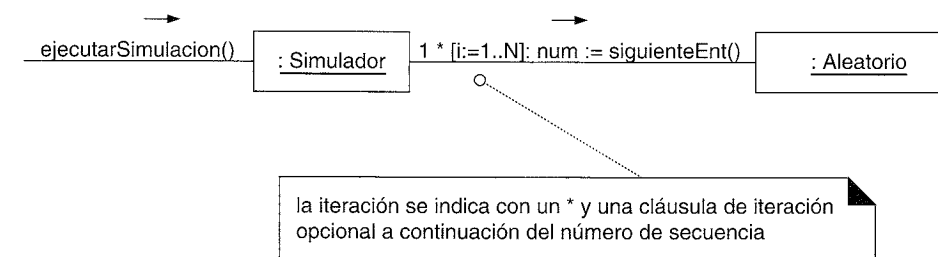


Figura 15.14. Iteración.

Iteración sobre una colección (multiobjeto)

Un algoritmo típico es iterar sobre todos los miembros de una colección (como una lista o una tabla), enviando un mensaje a cada uno de ellos. A menudo, se utiliza, finalmente, algún tipo de objeto iterador, como una implementación de *java.util.Iterator* o el iterador de la librería estándar de C++. En UML, el término **multiobjeto** se utiliza para denotar un conjunto de instancias —una colección—. En los diagramas de colaboración, se puede indicar como se muestra en la Figura 15.15.

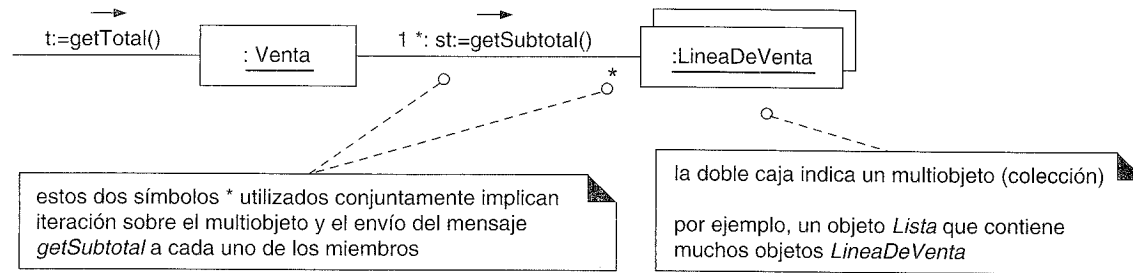


Figura 15.15. Iteración sobre un multiobjeto.

El marcador de multiplicidad “*” al final del enlace, se utiliza para indicar que se va a enviar el mensaje a cada elemento de la colección, en lugar de enviarse repetidamente al propio objeto colección.

Mensaje a un objeto clase

Los mensajes se podrían enviar a las propias clases, en lugar de una instancia, para invocar a los **métodos estáticos** o de clase. Se muestra un mensaje hacia el rectángulo de una clase cuyo nombre no está subrayado, lo que indica que el mensaje se está enviando a una clase en lugar de a una instancia (ver Figura 15.16).

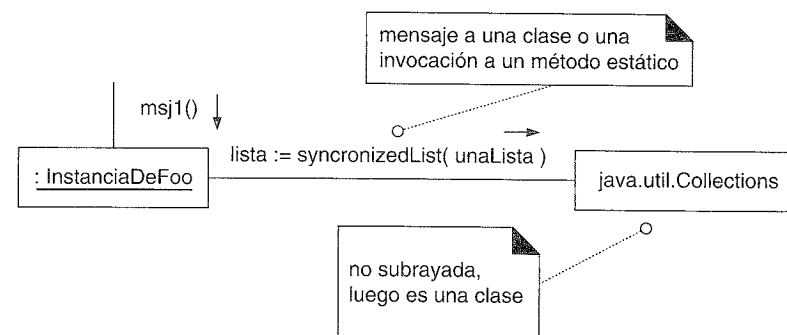


Figura 15.16. Mensaje a un objeto clase (invocación de un método estático).

En consecuencia es importante ser consistente y subrayar los nombre de las instancias cuando lo que se desea es una instancia, de otra manera, se podrían interpretar incorrectamente los mensajes a clases o instancias.

15.7. Notación básica de los diagramas de secuencia

Enlaces

A diferencia de los diagramas de colaboración, los diagramas de secuencia no muestran enlaces.

Mensajes

Cada mensaje entre objetos se representa con una expresión de mensaje sobre una línea con punta de flecha entre los objetos (ver Figura 15.17). El orden en el tiempo se organiza de arriba a abajo.

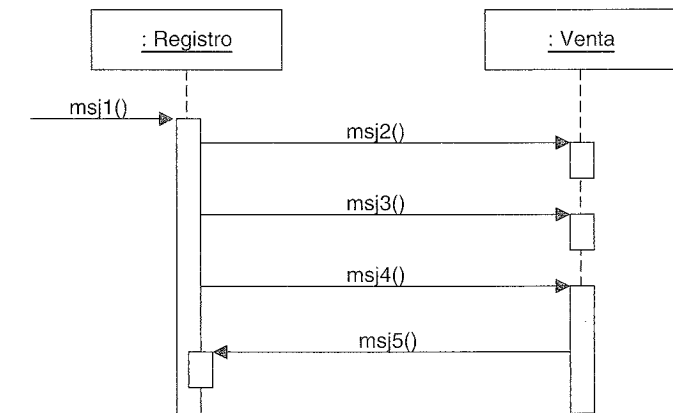


Figura 15.17. Mensajes y focos de control con cajas de activación.

Focos de control y cajas de activación

Como se ilustra en la Figura 15.17, los diagramas de secuencia podrían también mostrar los focos de control (esto es, en una llamada de rutina ordinaria, la operación se encuentra en la pila de llamadas) utilizando una **caja de activación**. La caja es opcional, pero la utilizan habitualmente los modeladores de UML.

Representación de retornos

Un diagrama de secuencia podría opcionalmente mostrar el retorno de un mensaje mediante una línea punteada con la punta de flecha abierta, al final de una caja de activación (ver Figura 15.18). Lo normal es que se excluya por quienes utilizan UML. Algunos anotan la línea de retorno para describir lo que se está devolviendo (si es el caso) a partir del mensaje.

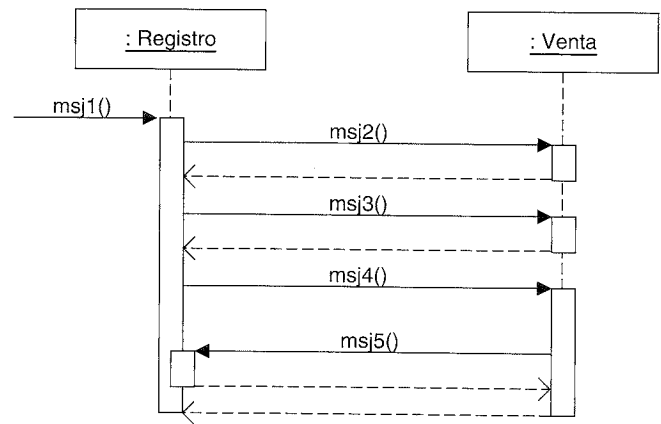


Figura 15.18. Representación de retornos.

Mensajes a “self” o “this”

Se puede representar un mensaje que se envía de un objeto a él mismo utilizando una caja de activación anidada (ver Figura 15.19).

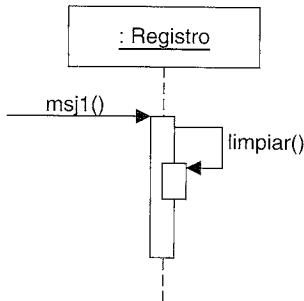


Figura 15.19. Mensajes a “this”.

Creación de instancias

La notación para la creación de instancias se muestra en la Figura 15.20.

Línea de vida del objeto y destrucción de objetos

La Figura 15.20 también ilustra las **líneas de vida de los objetos** —las líneas punteadas verticales bajo los objetos—. Éstas indican la duración de la vida de los objetos en el diagrama. En algunas circunstancias es deseable mostrar la destrucción explícita de un objeto (como en C++, que no tiene recogida de basura); la notación UML para las líneas de vida proporcionan una forma para expresar esta destrucción (ver Figura 15.21).

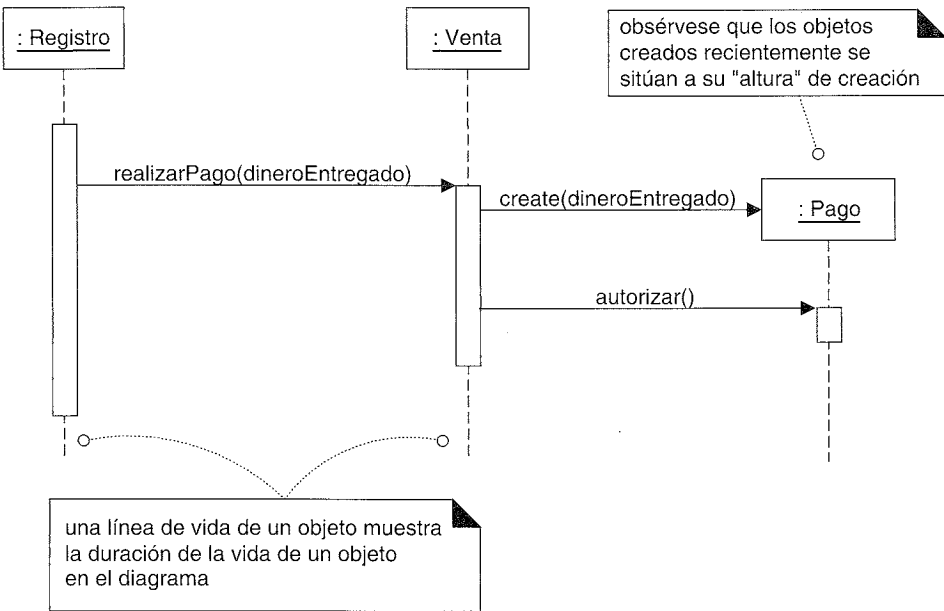


Figura 15.20. Creación de instancias y línea de vida de los objetos.

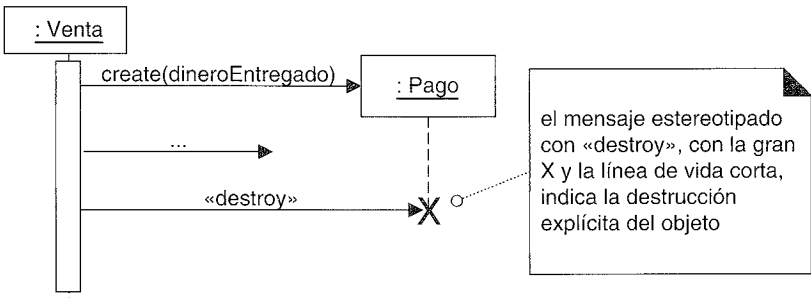


Figura 15.21. Destrucción de objetos.

Mensajes condicionales

La Figura 15.22 muestra un mensaje condicional.

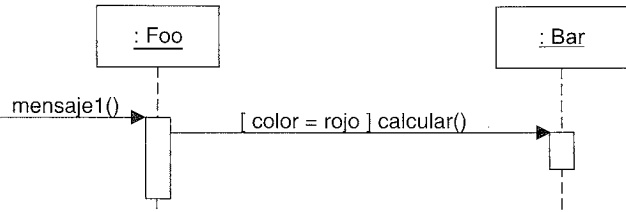


Figura 15.22. Un mensaje condicional.

Mensajes condicionales mutuamente exclusivos

La notación para este caso es un tipo de línea de mensaje con forma de ángulo que nace desde un mismo punto, como se ilustra en la Figura 15.23.

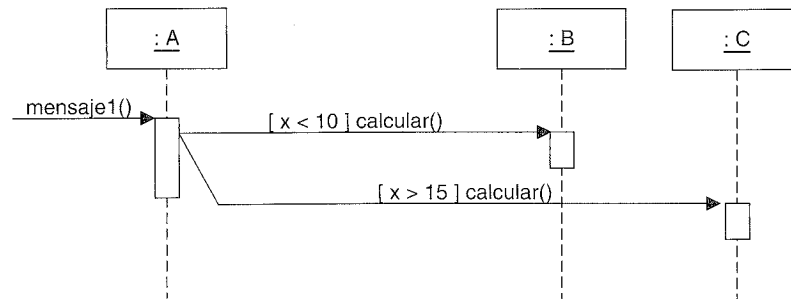


Figura 15.23. Mensajes condicionales mutuamente exclusivos.

Iteración para un único mensaje

La notación para la iteración de un mensaje se muestra en la Figura 15.24.

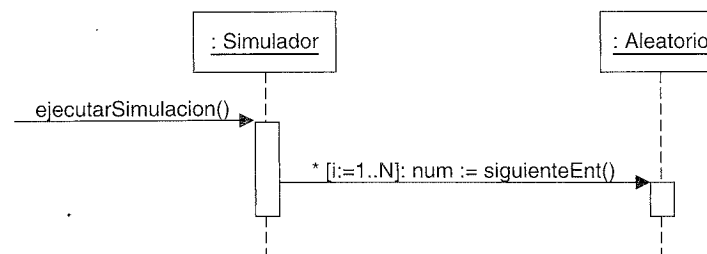


Figura 15.24. Iteración para un mensaje.

Iteración de una serie de mensajes

La Figura 15.25 muestra la notación para indicar la iteración alrededor de una serie de mensajes.

Iteración sobre una colección (multiobjeto)

En un diagrama de secuencia, la iteración sobre una colección se muestra en la Figura 15.26.

Con los diagramas de colaboración de UML, se especifica un marcador de multiplicidad '*' al final del rol (al lado del multiobjeto) para indicar el envío de un mensaje a cada elemento, en lugar de repetidamente a la propia colección. Sin embargo, UML no especifica cómo hacer esto con los diagramas de secuencia.

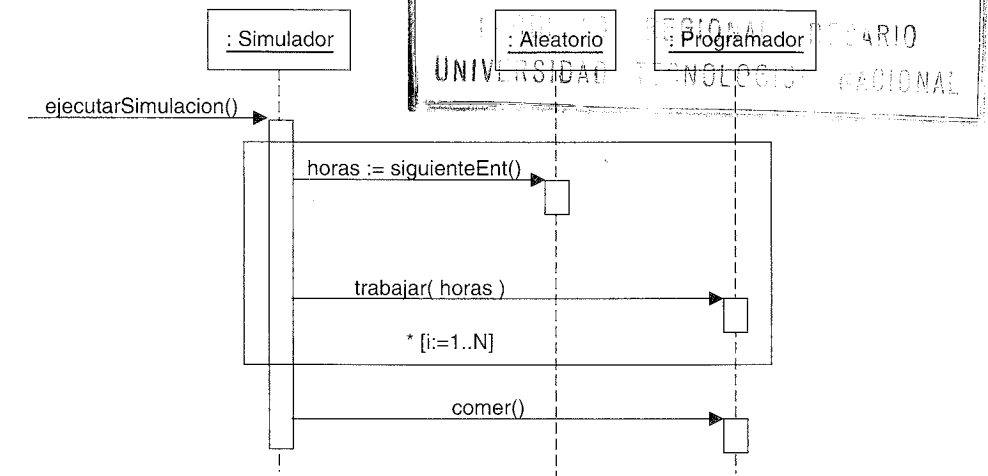


Figura 15.25. Iteración sobre una secuencia de mensajes.

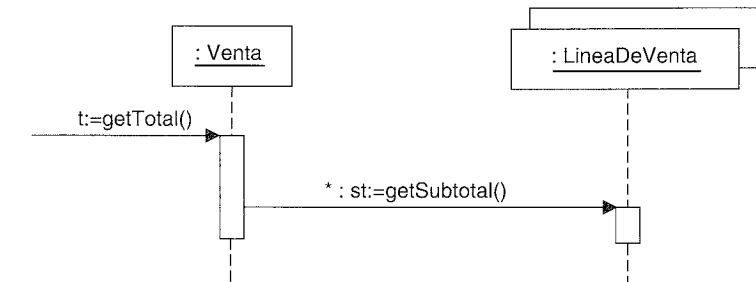


Figura 15.26. Iteración sobre un multiobjeto.

Mensajes a objetos de clase

Como en los diagramas de colaboración, las llamadas a los métodos de clase o estáticos se representan no subrayando el nombre del clasificador, lo que significa que se trata de una clase en lugar de una instancia (ver Figura 15.27).

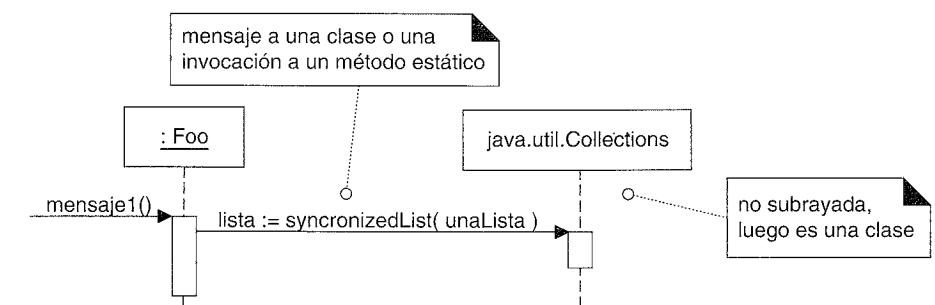
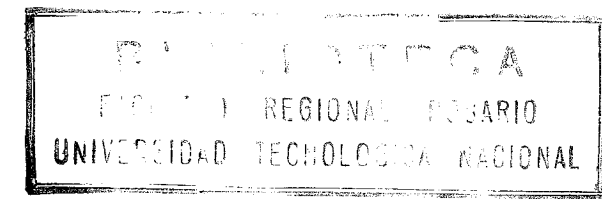


Figura 15.27. Invocación a un método de clase o estático.



GRASP: DISEÑO DE OBJETOS CON RESPONSABILIDADES

La forma más probable de que el mundo se destruya, coinciden la mayoría de los expertos, es por accidente. Aquí es donde entramos nosotros; somos profesionales informáticos. Nosotros provocamos accidentes.

Nathaniel Borenstein

Objetivos

- Definir los patrones.
 - Aprender a aplicar cinco de los patrones GRASP.
-

Introducción

El diseño de objetos se describe algunas veces como alguna variación de lo siguiente:

Después de la identificación de sus requisitos y la creación de un modelo del dominio, entonces añade métodos a las clases del software, y define el paso de mensajes entre los objetos para satisfacer los requisitos.

Un consejo tan conciso no es especialmente útil, porque existen profundos principios y cuestiones involucrados en estas etapas. La decisión de qué métodos, dónde colocarlos y cómo deberían interactuar los objetos, es muy importante y nada trivial. Hace falta una explicación cuidadosa, aplicable mientras se realizan los diagramas y la programación.

Y esta es una etapa crítica; es la parte esencial de lo que entendemos por desarrollar un sistema orientado a objetos, no el dibujo de diagramas del modelo del dominio, diagramas de paquetes, etcétera.

GRASP como un enfoque sistemático para aprender el diseño de objetos básico

Es posible comunicar los principios detallados y el razonamiento que se requiere para entender el diseño de objetos básico, y aprender a aplicarlos en un enfoque sistemático que elimina la magia y la ambigüedad.

Los patrones GRASP constituyen un apoyo para la enseñanza que ayuda a uno a entender el diseño de objetos esencial, y aplica el razonamiento para el diseño de una forma sistemática, racional y explicable. Este enfoque para la comprensión y utilización de los principios de diseño se basa en los *patrones de asignación de responsabilidades*.

16.1. Responsabilidades y métodos

UML define una **responsabilidad** como “un contrato u obligación de un clasificador” [OMG01]. Las responsabilidades están relacionadas con las obligaciones de un objeto en cuanto a su comportamiento. Básicamente, estas responsabilidades son de los siguientes dos tipos:

- Conocer.
- Hacer.

Entre las responsabilidades de **hacer** de un objeto se encuentran:

- Hacer algo él mismo, como crear un objeto o hacer un cálculo.
- Iniciar una acción en otros objetos.
- Controlar y coordinar actividades en otros objetos.

Entre las responsabilidades de **conocer** de un objeto se encuentran:

- Conocer los datos privados encapsulados.
- Conocer los objetos relacionados.
- Conocer las cosas que puede derivar o calcular.

Las responsabilidades se asignan a las clases de los objetos durante el diseño de objetos. Por ejemplo, podría declarar que “una *Venta* es responsable de la creación de una *LineaDeVenta*” (un hacer), o “una *Venta* es responsable de conocer su total” (un conocer). Las responsabilidades relevantes relacionadas con “conocer” a menudo se pueden inferir a partir del modelo del dominio, debido a los atributos y asociaciones que describe.

La granularidad de las responsabilidades influye en la conversión de las responsabilidades a clases y métodos. La responsabilidad de “proporcionar el acceso a bases de datos relacionales” podría implicar docenas de clases y cientos de métodos, empaquetados en un subsistema. En cambio, la responsabilidad de “crear una *Venta*” podría implicar sólo un método o unos pocos.

Una responsabilidad no es lo mismo que un método, pero los métodos se implementan para llevar a cabo responsabilidades. Las responsabilidades se implementan

utilizando métodos que o actúan solos o colaboran con otros métodos u objetos. Por ejemplo, la clase *Venta* podría definir uno o más métodos para conocer su total; digamos un método denominado *getTotal*¹. Para realizar esa responsabilidad, la *Venta* podría colaborar con otros objetos, por ejemplo, enviando el mensaje *getSubtotal* a cada objeto *LineaDeVenta* solicitando su subtotal.

16.2. Responsabilidades y los diagramas de interacción

El objetivo de este capítulo es ayudar a aplicar sistemáticamente los principios fundamentales para asignar responsabilidades a los objetos. A menudo, se hará durante la programación. En los artefactos UML, un contexto habitual donde se tiene en cuenta estas responsabilidades (implementadas como métodos) es durante la creación de los diagramas de interacción (que forman parte del Modelo de Diseño del UP), cuya notación básica se examinó en el capítulo anterior.

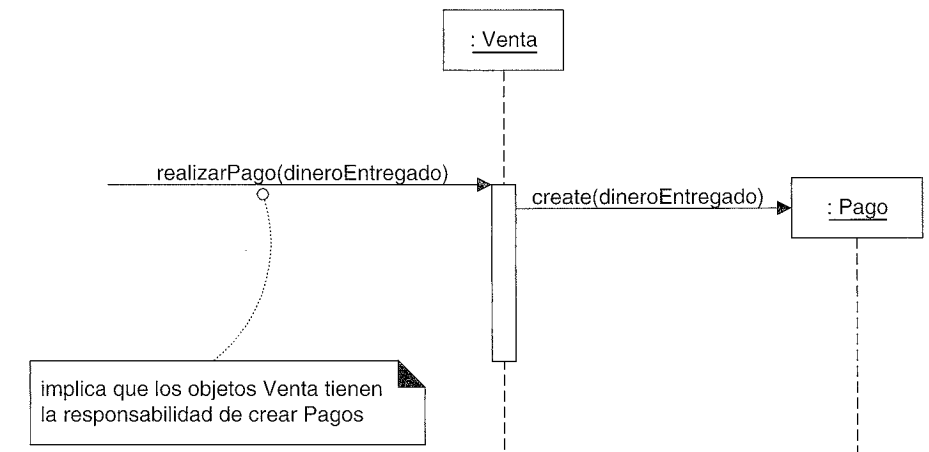


Figura 16.1. Las responsabilidades y los métodos están relacionados.

La Figura 16.1 indica que a los objetos *Venta* se les ha otorgado la responsabilidad de crear *Pagos*, que se invoca con el mensaje *realizarPago()* y se maneja con el correspondiente método *realizarPago*. Además, la realización de esta responsabilidad requiere colaboración para crear el objeto *LineaDeVenta* e invocar a su constructor.

En resumen, los diagramas de interacción muestran elecciones en la asignación de responsabilidades a los objetos. Cuando se crean, se han tomado las decisiones acerca de la asignación de responsabilidades, lo que se refleja en los mensajes que se envían a diferentes clases de objetos. Este capítulo describe con detalle los principios fundamentales —expresados en los patrones GRASP— para guiar las elecciones sobre dónde asignar responsabilidades. Estas elecciones se reflejan en los diagramas de interacción.

¹ N. del T.: Para nombrar los métodos de una clase que retornan el valor de cierta propiedad denominada <X> de dicha clase, utilizaremos la habitual terminología *get<X>*, tal y como comenta el propio autor en el Capítulo 17.

16.3. Patrones

Los desarrolladores orientados a objetos con experiencia (y otros desarrolladores de software) acumulan un repertorio tanto de principios generales como de soluciones basadas en aplicar ciertos estilos que les guían en la creación de software. Estos principios y estilos, si se codifican con un formato estructurado que describa el problema y la solución, y se les da un nombre, podrían llamarse **patrones**. Por ejemplo, a continuación presentamos un patrón de muestra:

Nombre del patrón:	Experto en Información
Solución:	Asignar una responsabilidad a la clase que tiene la información necesaria para cumplirla.
Problema que resuelve:	¿Cuál es el principio básico mediante el cual asignaremos responsabilidades a los objetos?

En la tecnología de objetos, un **patrón** es una descripción de un problema y la solución, a la que se da un nombre, y que se puede aplicar a nuevos contextos; idealmente, proporciona consejos sobre el modo de aplicarlo en varias circunstancias, y considera los puntos fuertes y compromisos². Muchos patrones proporcionan guías sobre el modo en el que deberían asignarse las responsabilidades a los objetos, dada una categoría específica del problema.

De manera más simple, un **patrón** es un par problema/solución con nombre que se puede aplicar en nuevos contextos, con consejos acerca de cómo aplicarlo en nuevas situaciones y discusiones sobre sus compromisos.

“Un patrón de una persona es un bloque de construcción primitivo de otra” es un dicho de la tecnología de objetos que ilustra la ambigüedad de lo que puede ser un patrón [GHJV94]. Este tratamiento de los patrones pasará por alto el hecho de qué es apropiado etiquetar como un patrón, y se centrará en el valor pragmático de utilizar el estilo de los patrones como un medio para nombrar, presentar, aprender y recordar principios de ingeniería del software útiles.

Patrones repetitivos

Un *nuevo patrón* podría considerarse un oxímoron, si describe una idea nueva. El término “patrón” tiene la intención de sugerir algo repetitivo. Lo importante de los patrones no es expresar nuevas ideas de diseño. Es cierto justamente lo contrario —los patrones pretenden codificar conocimiento, estilos y principios *existentes* y que se han probado que son válidos—; cuanto más trillados y extendidos, mejor.

² La noción formal de patrones nació con los patrones arquitectónicos (de construcción) de Christofer Alexander [AIS77]. Los patrones de software se originaron en los ochenta con Kent Beck quien, llegó a ser consciente del trabajo con patrones de Alexander en arquitectura, y entonces los desarrolló junto con Ward Cunningham [BC87, Beck94].

En consecuencia, los patrones GRASP —que pronto se introducirán— no establecen nuevas ideas: son una codificación de principios básicos ampliamente utilizados. Para un experto en objetos, los patrones GRASP —por su idea, y no por su nombre— parecerán muy elementales y familiares. ¡Eso es lo importante!

Los patrones tienen nombres

Todos los patrones, idealmente, tienen nombres sugerentes. El asignar un nombre a un patrón, técnica o principio tiene las siguientes ventajas:

- Apoya la identificación e incorporación de ese concepto en nuestro conocimiento y memoria.
- Facilita la comunicación.

Asignar un nombre a una idea compleja como un patrón es un ejemplo del poder de la abstracción —reducción de una forma compleja a una simple eliminando detalles—. Por tanto, los patrones GRASP tienen nombres concisos como *Experto en Información*, *Creador*, *Variaciones Protegidas*.

La asignación de nombres a los patrones mejora la comunicación

Cuando se asigna un nombre a un patrón, podemos discutir con otros un principio complejo o una idea de diseño con un nombre sencillo. Considere la siguiente conversación entre dos diseñadores de software, que utilizan un vocabulario de patrones común (*Creador*, *Factoría*, etcétera), para tomar una decisión sobre un diseño:

Alfredo: “¿Dónde crees que deberíamos colocar la responsabilidad de crear una *LineaDeVenta*? Creo en que una *Factoría*.”

Teresa: “Siguiendo el *Creador*, creo que la *Venta* sería adecuada.”

Alfredo: “Oh, correcto. Estoy de acuerdo.”

Identificar los estilos y principios de diseño con nombres sobreentendidos comúnmente facilita la comunicación y eleva el nivel de la investigación a un grado de abstracción más alto.

16.4. GRASP: Patrones de Principios Generales para Asignar Responsabilidades

Resumiendo la introducción anterior:

- La asignación habilidosa de responsabilidades es extremadamente importante en el diseño de objetos.
- La decisión acerca de la asignación de responsabilidades, a menudo, tiene lugar durante la creación de los diagramas de interacción y con seguridad durante la programación.

- Los patrones son pares problema/solución con un nombre que codifican buenos consejos y principios relacionados con frecuencia con la asignación de responsabilidades.

Pregunta: ¿Qué son los patrones GRASP?

Respuesta: Describen los principios fundamentales del diseño de objetos y la asignación de responsabilidades, expresados como patrones.

Es importante entender y ser capaces de aplicar estos principios durante la creación de los diagramas de interacción porque un desarrollador de software con poca experiencia en la tecnología de objetos necesita dominar estos principios tan rápido como sea posible; constituyen la base de cómo se diseñará el sistema.

GRASP es un acrónimo de **General Responsibility Assignment Software Patterns** (patrones generales de software para asignar responsabilidades)³. El nombre se eligió para sugerir la importancia de *aprehender* (*grasping* en inglés) estos principios para diseñar con éxito el software orientado a objetos.

Cómo aplicar los patrones GRASP

Las secciones siguientes presentan los primeros cinco patrones GRASP:

- Experto en Información.
- Creador.
- Alta Cohesión.
- Bajo Acoplamiento.
- Controlador.

Hay otros, que se introducirán en un capítulo posterior, pero merece la pena dominar estos cinco primeros porque se refieren a cuestiones muy básicas, comunes y a aspectos fundamentales del diseño.

Por favor, estudie los siguientes patrones, observe cómo se utilizan en los diagramas de interacción de ejemplo, y entonces aplíquelos durante la creación de nuevos diagramas de interacción. Comience dominando *Experto en Información*, *Creador*, *Controlador*, *Alta Cohesión* y *Bajo Acoplamiento*. Después aprenda los patrones restantes.

16.5. Notación del diagrama de clases UML

Un rectángulo de clase de UML, que se utiliza para representar las clases software, normalmente muestra tres compartimentos; el tercero representa los métodos de la clase, como se muestra en la Figura 16.2.

³ Técnicamente deberíamos escribir “Patrones GRAS” en lugar de “Patrones GRASP”, pero lo segundo suena mejor.

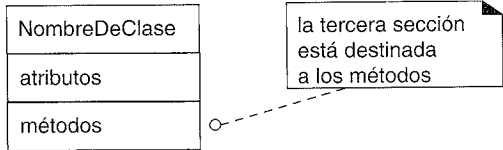


Figura 16.2. Las clases software muestran los nombres de los métodos.

Los detalles de esta notación se estudian en un capítulo siguiente. En la siguiente discusión sobre patrones, se utilizará ocasionalmente esta forma de rectángulo de clase.

16.6. Experto en Información (o Experto)

Solución Asignar una responsabilidad al experto en información —la clase que tiene la *información* necesaria para realizar la responsabilidad—.

Problema ¿Cuál es un principio general para asignar responsabilidades a los objetos?

Un Modelo de Diseño podría definir cientos o miles de clases software, y una aplicación podría requerir que se realicen cientos o miles de responsabilidades. Durante el diseño de objetos, cuando se definen las interacciones entre los objetos, tomamos decisiones sobre la asignación de responsabilidades a las clases software. Si se hace bien, los sistemas tienden a ser más fáciles de entender, mantener y ampliar, y existen más oportunidades para reutilizar componentes en futuras aplicaciones.

Ejemplo En la aplicación del PDV NuevaEra, algunas clases necesitan conocer el total de una venta.

Comience asignando responsabilidades estableciendo claramente la responsabilidad.

Siguiendo este consejo, la pregunta es:

¿Quién debería ser el responsable de conocer el total de una venta?

Siguiendo el *Experto en Información*, deberíamos buscar las clases de objetos que contienen la información necesaria para determinar el total.

Ahora llegamos a una pregunta clave: ¿miramos en el Modelo del Dominio o en Modelo de Diseño para analizar las clases que tienen la información necesaria? El Modelo del Dominio representa clases conceptuales del dominio del mundo real; el Modelo de Diseño representa clases software.

Respuesta:

1. Si hay clases relevantes en el Modelo del Diseño, mire ahí primero.
2. Si no, mire en el Modelo del Dominio, e intente utilizar (o ampliar) sus representaciones para inspirar la creación de las correspondientes clases de diseño.

Por ejemplo, asuma que acabamos de comenzar el trabajo de diseño y no hay más que un Modelo de Diseño mínimo. Por tanto, miramos en el Modelo del Dominio buscando expertos en información; quizás la *Venta* del mundo real es uno. Entonces, aña-

dimos una clase software al Modelo de Diseño denominada, igualmente, *Venta*, y le otorgamos la responsabilidad de conocer su total, representado con el método llamado *getTotal*. Este enfoque mantiene un *salto en la representación bajo* en el que el diseño de los objetos software se corresponde con nuestra concepción sobre cómo se organiza el mundo real.

Para examinar este caso en detalle, considere el Modelo del Dominio parcial de la Figura 16.3.

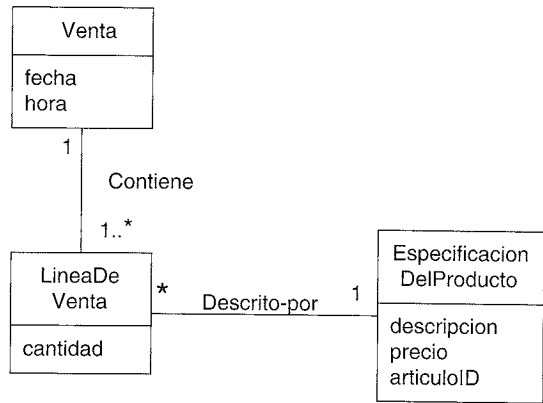


Figura 16.3. Asociaciones de Venta.

¿Qué información se necesita para determinar el total? Es necesario conocer todas las instancias de *LineaDeVenta* de una venta y la suma de sus subtotales. Una instancia de *Venta* las contiene; por tanto, por la guía del Experto en Información, la clase de objeto *Venta* es adecuada para esta responsabilidad; es un *experto en información* para el trabajo.

Como se mencionó, es en el contexto de la creación de los diagramas de interacción cuando surgen estas cuestiones de responsabilidad. Imagine que estamos comenzando a trabajar con detalle en la elaboración de los diagramas de interacción para asignar responsabilidades a los objetos. Los diagramas de interacción y de clases parciales de la Figura 16.4 ilustran algunas decisiones.

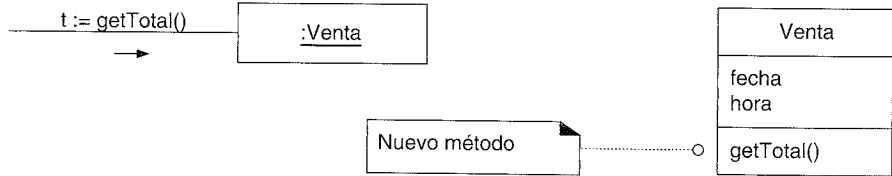


Figura 16.4. Diagramas de interacción y de clases parciales.

Todavía no hemos terminado. ¿Qué información se necesita para determinar el subtotal de la línea de venta? Se necesitan *LineaDeVenta.cantidad* y *EspecificacionDelProducto.precio*. La *LineaDeVenta* conoce su cantidad y la *EspecificacionDelProducto* asociada; por tanto, siguiendo el patrón Experto, la *LineaDeVenta* debería determinar el subtotal; es el *experto en información*.

En cuanto a los diagramas de interacción, esto significa que la *Venta* necesita enviar el mensaje *getSubtotal* a cada una de las *LineaDeVenta* y sumar el resultado; este diseño se muestra en la Figura 16.5.

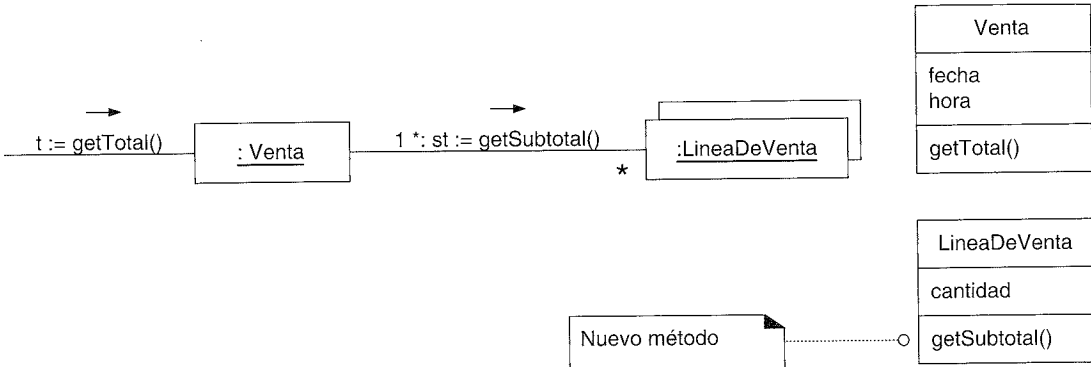


Figura 16.5. Cálculo del total de la Venta.

Una *LineaDeVenta* para realizar la responsabilidad de conocer y proporcionar su subtotal necesita conocer el precio del artículo.

La *EspecificacionDelProducto* es un experto en información que proporciona su precio; por tanto, se le debe enviar un mensaje solicitando su precio.

El diseño se muestra en la Figura 16.6.

En conclusión, para realizar la responsabilidad de conocer y proporcionar el total de una venta, se asignaron tres responsabilidades a tres clases de diseño de objetos como sigue.

Clase de Diseño	Responsabilidad
Venta	Conocer el total de la venta.
LineaDeVenta	Conocer el subtotal de la línea de venta.
EspecificacionDelProducto	Conocer el precio del artículo.

El contexto en el que se consideraron y optaron por estas responsabilidades fue durante la elaboración de un diagrama de interacción. La sección de métodos de un diagrama de clases puede entonces incluir los métodos.

El principio mediante el que se asignó cada responsabilidad fue el Experto en Información —colocándola en el objeto que tiene la información necesaria para realizarla—.

El Experto en Información se utiliza con frecuencia en la asignación de responsabilidades; es un principio de guía básico que se utiliza continuamente en el diseño de objetos. El Experto no pretende ser una idea oscura o extravagante; expresa la “intuición” común de que los objetos hacen las cosas relacionadas con la información que tienen.

Nótese que el cumplimiento de la responsabilidad a menudo requiere información que se encuentra dispersa por diferentes clases de objetos. Esto implica que hay muchos expertos en información “parcial” que colaborarán en la tarea. Por ejemplo, el problema del total de las ventas al final requiere la colaboración de tres clases de objetos. Cada vez

Discusión